SHADERMESH: A DECENTRALIZED MARKETPLACE FOR RENTING GPU AND LLM COMPUTE ON ETHEREUM

A Project Report

submitted by

RAKESH KUMAR RABHA

(24BCC7002)

[MEMBER 2 NAME]

([MEMBER 2 REG. NO.])

[MEMBER 3 NAME]

([MEMBER 3 REG. NO.])

in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Under the guidance of [GUIDE NAME], [GUIDE DESIGNATION]



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

VIT-AP UNIVERSITY, AMARAVATI, ANDHRA PRADESH

OCTOBER 2026

DECLARATION BY THE CANDIDATES

We hereby declare that the project report entitled "**ShaderMesh: A Decentralized Marketplace for Renting GPU and LLM Compute on Ethereum**" submitted by us to VIT-AP University, Amaravati in partial fulfilment of the requirement for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering** is a record of bonafide project work carried out by us under the supervision of **[GUIDE NAME], [GUIDE DESIGNATION]**. We further declare that the work reported in this report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Amaravati

Date:

Signature of the student
Name: Rakesh Kumar Rabha
Reg. Number: 24BCC7002

Signature of the student
Name: [MEMBER 2 NAME]
Reg. Number: [MEMBER 2 REG. NO.]

Signature of the student
Name: [MEMBER 3 NAME]
Reg. Number: [MEMBER 3 REG. NO.]

School of Computer Science and Engineering

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "**ShaderMesh: A Decentralized Marketplace for Renting GPU and LLM Compute on Ethereum**" submitted by Rakesh Kumar Rabha (24BCC7002); [MEMBER 2 NAME] ([MEMBER 2 REG. NO.]); [MEMBER 3 NAME] ([MEMBER 3 REG. NO.]) to VIT-AP University, Amaravati in partial fulfilment of the requirement for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering** is a record of bonafide project work undertaken by them under my supervision. The work fulfils the requirements as per the regulations of this Institute and, in my opinion, meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Signature of the Guide
Name: [GUIDE NAME]
GUIDE
[GUIDE DESIGNATION]
Date:

Signature of the Head of the Department
Name: [HOD NAME]
HEAD OF THE DEPARTMENT
School of Computer Science and Engineering
Date:

Signature
Internal Examiner(s)
Date:

Signature
External Examiner(s)
Date:

ACKNOWLEDGEMENT

We thank our guide, [GUIDE NAME], [GUIDE DESIGNATION], School of Computer Science and Engineering, VIT-AP University, for guidance and review during the project, and for the feedback at Review 1 that shaped the move from a rendering marketplace to a compute-rental design. We thank the Head of the Department, [HOD NAME], and the faculty of the School of Computer Science and Engineering for the academic environment in which this work was done. We also acknowledge the maintainers of the open-source projects on which the system is built, including OpenZeppelin, Hardhat, Next.js, Ollama, IPFS, and PostgreSQL, whose documentation made it possible to build and test the system in the time available.

Place: Amaravati
Date:

Rakesh Kumar Rabha (24BCC7002)
[MEMBER 2 NAME] ([MEMBER 2 REG. NO.])
[MEMBER 3 NAME] ([MEMBER 3 REG. NO.])

ABSTRACT

Demand for GPU compute has grown with the spread of large language models, while many capable GPUs sit idle. Renting capacity between strangers needs payment security, evidence of service, and enforcement of penalties, and these are normally supplied by a trusted marketplace operator. This project builds ShaderMesh, a decentralized marketplace for renting GPU and CPU compute on Ethereum, in which the blockchain is the trust, coordination, and payment layer and all compute runs off chain.

ShaderMesh uses two explicit trust models over one staking, reputation, and slashing layer. Rentals cannot be re-executed and checked, so they are secured by streamed escrow, bonded provider stake, strict per-interval signed heartbeats, hardware attestation, and seeded benchmark challenges that slash free stake on failure. Deterministic jobs, which can be recomputed, use an optimistic verification game with spot checks and slashing. Managed LLM inference is sold through prepaid credit and an OpenAI-compatible gateway with signed requests.

The system consists of six Solidity contracts, a Python provider agent, an attester service, a TypeScript orchestrator with a rebuildable Postgres projection, a Next.js web application, and a Monte Carlo simulator. It runs locally with Docker Compose and is deployed and source-verified on the Sepolia test network. The automated suites contain 42 contract tests, 58 agent tests, 54 orchestrator tests, 15 simulator tests, and 24 browser checks that include wallet writes.

The evaluation reports gas for 27 core transactions, a simulation that reproduces the break-even spot-check rate of 0.5 and confirms conservation of escrow in rentals, and a live run on an RTX 4070 Laptop GPU provider that registered, staked, served a real model, and passed a live benchmark challenge with a score of 28,919. The

simulation also shows that detection falls below the nominal spot-check rate in a mixed challenger pool, and that strict heartbeat termination penalizes brief outages. The report states the limits of the design: the attester is a trusted judge, LLM output is not verified on chain, and a heartbeat proves the liveness of a key and not the quality of service.

TABLE OF CONTENTS

ACKNOWLEDGEMENT iv

ABSTRACT v

LIST OF TABLES xiii

LIST OF FIGURES xv

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE xvii

CHAPTER 1: INTRODUCTION 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Why Blockchain 2

1.4 Scope of the Project 3

1.5 Organization of the Report 4

CHAPTER 2: OBJECTIVES AND MOTIVATION 5

2.1 Objectives 5

2.2 Motivation 5

2.2.1 Economic 5

2.2.2 Technical 6

2.2.3 Educational 6

2.2.4 Practical 6

2.3 Design Principles 6

CHAPTER 3: LITERATURE SURVEY 7

3.1 Programmable Settlement 7

3.2 Decentralized Compute Marketplaces 7

3.3 Verification of Off-Chain Computation 8

3.4 LLM Serving 8

3.5 Comparison with ShaderMesh 8

3.6 Synthesis 9

CHAPTER 4: EXISTING SYSTEM AND IDENTIFIED GAP 10
4.1 Current Methods 10
4.2 Problems and Limitations 10
4.3 Identified Gap 11
4.4 Proposed Approach 11
CHAPTER 5: REQUIREMENTS 13
5.1 Actors 13
5.2 Functional Requirements 13
5.2.1 Rental market 14
5.2.2 Inference 14
5.2.3 Verifiable tasks 15
5.2.4 Provider operations and attestation 15
5.3 Non-Functional Requirements 15
5.4 Traceability and Status 16
CHAPTER 6: SYSTEM DESIGN 18
6.1 Architecture Overview 18
6.2 Trust Boundary 19
6.3 On-Chain and Off-Chain Boundary 20
6.4 Data Model 20
6.5 Token Economics 21
6.5.1 Stake, bond, and slash 21
6.5.2 Reputation 22
6.5.3 Protocol parameters 22
6.5.4 Deterrence for verifiable tasks 23
6.5.5 Deterrence for rentals 23
6.6 State Machines 23
CHAPTER 7: SMART CONTRACT DESIGN 25

7.1 SmeshToken 25
7.2 ProviderRegistry 25
7.2.1 State 25
7.2.2 Key functions 25
7.2.3 Events and access control 26
7.2.4 Invariants 26
7.3 RentalMarket 27
7.3.1 State 27
7.3.2 Key functions 27
7.3.3 Heartbeat rule 28
7.3.4 Invariants 28
7.4 InferenceMarket 29
7.5 JobMarket 29
7.6 Verification 30
7.7 Contract Function Summary 31
7.8 Benchmark Challenge Mechanism 31
7.8.1 Protocol 31
7.8.2 Security reasoning 33
CHAPTER 8: IMPLEMENTATION 35
8.1 Technology Stack 35
8.2 Provider Agent 35
8.2.1 Lease serving 36
8.2.2 Sandboxing 36
8.2.3 Inference serving 36
8.2.4 Benchmark responder 37
8.3 Attester Service 37
8.4 Orchestrator 37

8.4.1 Event index and persistence 37
8.4.2 Inference gateway 38
8.4.3 Split and stitch 38
8.5 Web Application 38
8.5.1 Pages 39
8.5.2 Design system 39
8.5.3 Wallet writes 40
8.6 Deployment 41
8.6.1 Local stack with Docker Compose 41
8.6.2 Provider-only node 41
8.6.3 Sepolia operator and provider topology 41
CHAPTER 9: TESTING 43
9.1 Strategy 43
9.2 Contract Tests 43
9.3 Provider Agent Tests 44
9.4 Orchestrator Tests 44
9.5 Simulator Tests 44
9.6 End-to-End Browser Tests 45
9.6.1 Wallet-write automation 45
9.7 Continuous Integration 45
9.8 Gaps 46
CHAPTER 10: RESULTS AND EVALUATION 47
10.1 Evaluation Setup 47
10.2 Gas Costs 47
10.3 Verifiable-Task Simulation 49
10.3.1 Break-even spot-check rate 49
10.3.2 Mixed provider population 50

10.4 Rental Simulation 51
10.5 Live Sepolia Results 53
10.5.1 Deployment 53
10.5.2 Provider and challenge 53
10.5.3 Flows validated on the Review 1 deployment 55
10.5.4 Application screens 55
10.6 Summary Against Objectives 58
CHAPTER 11: LIMITATIONS, RISKS, AND ETHICAL CONSIDERATIONS 60
11.1 Limitations 60
11.1.1 The attester is a trusted judge 60
11.1.2 LLM output is not verified on chain 60
11.1.3 A heartbeat proves liveness of a key, not quality of service 60
11.1.4 A benchmark does not prove the GPU model 60
11.1.5 Requester-metered inference 61
11.1.6 Single live provider and testnet only 61
11.1.7 Task verification is not fully hardened 61
11.1.8 Other limits 61
11.2 Risks 61
11.3 Ethical Considerations 62
CHAPTER 12: CONCLUSION AND FUTURE WORK 64
12.1 Conclusion 64
12.2 Future Work 64
12.2.1 Protocol 64
12.2.2 Verification of workloads 65
12.2.3 Platform 65
12.2.4 Assurance 65
REFERENCES 66

APPENDIX A: DEPLOYMENT ADDRESSES AND REPRODUCTION 69

APPENDIX B: API ENDPOINT SUMMARY 70

APPENDIX C: TEST COMMANDS 71

LIST OF TABLES

TABLE 3.1: Comparison of related systems with ShaderMesh 9

TABLE 5.1: Functional requirement groups 14

TABLE 5.2: Non-functional requirements 15

TABLE 5.3: Requirements traceability and implementation status 16

TABLE 6.1: Placement of data and logic across the trust boundary 20

TABLE 6.2: Main entities and key fields 21

TABLE 6.3: Default protocol parameters 22

TABLE 6.4: Verifiable task states and transitions 24

TABLE 7.1: ProviderRegistry functions 26

TABLE 7.2: RentalMarket functions 27

TABLE 7.3: InferenceMarket functions 29

TABLE 7.4: JobMarket functions 30

TABLE 7.5: Summary of contract surface and access control 31

TABLE 8.1: Technology stack 35

TABLE 8.2: Provider agent modules 35

TABLE 8.3: Web application routes 39

TABLE 8.4: Sepolia deployment topology 42

TABLE 9.1: Test suite summary 43

TABLE 10.1: Gas used by core transactions 47

TABLE 10.2: Expected value of cheating by spot-check rate (reward 100, bond 100) 50

TABLE 10.3: Task verification outcomes in a mixed population 51

TABLE 10.4: Simulated rental outcomes by provider type 52

TABLE 11.1: Risk register 62

TABLE A.1: Sepolia contract addresses 69

TABLE B.1: Orchestrator API endpoints 70

TABLE C.1: Commands to run each test suite..... 71

LIST OF FIGURES

Fig. 4.1: System context: actors, ShaderMesh, and the external systems it depends on 11

Fig. 6.1: Container view: off-chain components (untrusted) and the contracts that hold all value..... 18

Fig. 6.2: What each component can and cannot do 19

Fig. 6.3: Token flows: deposits, per-interval payouts, refunds, and slashes 22

Fig. 6.4: Lease states and the treatment of escrow and bond at each terminal state.. 24

Fig. 7.1: Rental lifecycle: listing, lease, heartbeats, settlement, and the two ways a lease ends 28

Fig. 7.2: Benchmark challenge: issue, answer, judge, and the permissionless expiry paths 32

Fig. 7.3: Benchmark challenge states and transition conditions..... 33

Fig. 8.1: Landing page in the light theme 40

Fig. 8.2: Landing page in the dark theme 40

Fig. 10.1: Gas per core transaction, with benchmark challenge calls highlighted 48

Fig. 10.2: Expected cheater profit against spot-check probability (verifiable task mode) 50

Fig. 10.3: Simulated rental outcomes by provider type: paid, refunded, and slashed 52

Fig. 10.4: Provider console on Sepolia: the RTX 4070 Laptop GPU provider with 100 SMESH of stake, attested, one challenge passed..... 54

Fig. 10.5: Explorer Challenges tab on Sepolia: benchmark challenge #1 passed with score 28919, answered in 108 s 54

Fig. 10.6: Overview page with live network figures 56

Fig. 10.7: Rent GPUs page listing available capacity..... 56

Fig. 10.8: Models page with the inference offers from the live provider 57

Fig. 10.9: Explorer Leases tab with the lease inspector..... 57

Fig. 10.10: Explorer events and slashes timeline..... 58

Fig. 10.11: Explorer Challenges tab in the dark theme 58

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

Abbreviations

Abbreviation	Expansion
API	Application Programming Interface
CI	Continuous Integration
CID	Content Identifier (IPFS)
EIP	Ethereum Improvement Proposal
EMA	Exponential Moving Average
ERC	Ethereum Request for Comments (token standard)
ETH	Ether, the native currency of Ethereum
EVM	Ethereum Virtual Machine
GPU	Graphics Processing Unit
IPFS	InterPlanetary File System
JSON-RPC	JavaScript Object Notation Remote Procedure Call
LLM	Large Language Model
RPC	Remote Procedure Call
SHA-256	Secure Hash Algorithm, 256 bit
SLA	Service Level Agreement
SMESH	ShaderMesh utility token
SSE	Server-Sent Events
SSIM	Structural Similarity Index
TEE	Trusted Execution Environment
WCAG	Web Content Accessibility Guidelines

Symbols and Nomenclature

- **R, B:** reward per task and bond per task, in SMESH.
- **s, s*:** spot-check probability and its break-even value, $s^* = R / (R + B)$.
- **V, D, h, h*:** value captured per interval by a false claim, bond at risk, detection probability, and break-even detection $h^* = V / (V + D)$.
- **Reputation:** integer from 0 to 10,000 updated by an exponential moving average with $K = 5$.
- **Provider, requester, attester:** the roles defined in Chapter 5.

CHAPTER 1: INTRODUCTION

1.1 Background

Demand for graphics processing unit (GPU) compute has grown quickly with the spread of machine learning and large language models (LLMs). Training, fine-tuning, and serving a model all need accelerators, and most of the capacity that can be rented on demand sits with a small number of cloud operators. At the same time, many capable GPUs in workstations, studios, and laboratories are idle for most of the day. The two facts point to a market that is hard to organize: owners of spare capacity have no simple way to rent it to strangers, and renters have no simple way to trust capacity they cannot inspect.

A conventional marketplace solves this with a trusted operator. The operator verifies listings, holds payment, meters usage, and settles disputes. That arrangement works, but it concentrates power. The operator can change prices, suspend accounts, and decide dispute outcomes, and the billing and reliability history of every participant lives in a database that only the operator controls. Public blockchains offer a different foundation: a shared ledger that holds funds under rules that no single party can rewrite [1], [2].

ShaderMesh is a decentralized marketplace for renting GPU and CPU compute on Ethereum. Providers stake a utility token and rent out capacity. Requesters lease that capacity to run containerized workloads, or buy managed, OpenAI-compatible LLM inference from a provider [3]. The blockchain is the trust, coordination, and payment layer. It is never the compute layer: workloads, model inference, indexing, and file storage all run off chain.

1.2 Problem Statement

Renting compute from a stranger raises four problems that a marketplace must address.

1. **Payment under mutual distrust.** The renter does not want to pay before service is delivered. The provider does not want to deliver before payment is secured.
2. **Proof of service.** A rented machine runs a workload that cannot be re-executed and compared. An LLM response or a training run has no single correct answer

to check, so the market needs evidence that the service was available, not proof that the result was right.

3. **Proof of capability.** A provider can advertise a GPU it does not have. The market needs a way to test advertised hardware repeatedly and to penalize false claims.
4. **Neutral enforcement.** Penalties for missed service, such as refunds and loss of collateral, must execute automatically under public rules, without an operator who can favor one side.

A second class of work has the opposite property. Deterministic jobs, such as rendering a tile of an image or running a fixed batch computation, produce an output that another machine can recompute and compare. For these jobs the market can use verification by re-execution. A single design that treats both classes the same way would either over-verify rentals, which is impossible, or under-verify deterministic jobs, which is wasteful.

1.3 Why Blockchain

A blockchain is justified here only where participants who do not trust each other need shared, tamper-resistant state and automatic enforcement. ShaderMesh uses Ethereum for exactly these functions.

- **Escrow with streamed settlement.** A contract holds the renter funds and releases them per completed interval, then refunds the rest.
- **Staking and slashing.** Providers lock collateral that the contracts can confiscate for provable failures, with no operator discretion.
- **On-chain reputation.** Outcomes are recorded in a public score that no marketplace database controls.
- **Token payments.** A single ERC-20 token [4] settles payments and collateral without banks or chargebacks.
- **Auditable rules.** Pricing, leasing, attestation, and penalty logic are open code that any participant can read and verify.

Ethereum cannot observe a GPU, judge an LLM answer, or run a heavy workload at acceptable cost. The project therefore keeps the chain small. It stores money, collateral, lifecycle state, commitments, and content identifiers, and leaves

everything heavy to untrusted off-chain components that cannot move funds or force a result. The base-layer security is inherited from Ethereum proof of stake [5]. The project contributes application-layer mechanism design on top of it.

1.4 Scope of the Project

The delivered system is a working research and testnet prototype. It covers the following.

- Six Solidity contracts (SmeshToken, ProviderRegistry, RentalMarket, InferenceMarket, JobMarket, and Verification) with 42 automated tests.
- A provider agent in Python that serves leases, serves inference, executes deterministic tasks in constrained Docker containers, sends heartbeats, and answers benchmark challenges.
- An attester service that records hardware attestations and issues, judges, and expires benchmark challenges.
- An orchestrator in TypeScript that indexes chain events into a rebuildable Postgres projection, serves a read API and an inference gateway, and splits and stitches deterministic jobs through IPFS.
- A Next.js web application with a landing page, a dashboard, rental, model, provider, and explorer pages, in light and dark themes.
- A Monte Carlo simulator of the verification game and of rental economics.
- A local deployment with Docker Compose and a public deployment on the Sepolia test network [6] with source-verified contracts.

The following items are outside the scope of this project. They are stated here so the results in later chapters are read correctly.

- A new base-layer blockchain or consensus algorithm.
- On-chain verification of LLM output correctness, or on-chain training.
- Production custody, identity verification, or fiat payment rails.
- A token sale or any real financial instrument. SMESH is a utility token that has value only on test networks.
- Blender rendering with perceptual verification, which was descoped to future work in the final phase plan.

1.5 Organization of the Report

Chapter 2 states the objectives and motivation. Chapter 3 surveys related work and compares it with ShaderMesh, and Chapter 4 describes existing systems and the gap that remains. Chapter 5 summarizes the requirements and their implementation status. Chapter 6 presents the system design, and Chapter 7 describes each smart contract and the benchmark challenge mechanism in detail. Chapter 8 covers the implementation of the off-chain components and the deployment. Chapter 9 describes testing. Chapter 10 reports the evaluation results, including gas costs, simulation results, and the live Sepolia deployment. Chapter 11 states limitations, risks, and ethical considerations, and Chapter 12 concludes with future work. The appendices list deployment addresses, the application programming interface (API), and the test commands.

CHAPTER 2: OBJECTIVES AND MOTIVATION

2.1 Objectives

The project had five objectives, fixed at the Review 1 stage [7]. Each is listed below with the outcome measured at the end of the project.

1. **Design a decentralized compute-rental marketplace on Ethereum** with on-chain listings, leases, streamed escrow, provider staking, and settlement by smart contract. Outcome: the RentalMarket, InferenceMarket, and ProviderRegistry contracts implement these functions, and a lease was run end to end on Sepolia.
2. **Secure rented compute without result verification** using stake, signed uptime heartbeats, hardware attestation and benchmark challenges, reputation, and slashing for service-level breaches. Outcome: implemented and tested. Benchmark challenges run live on Sepolia.
3. **Specialize in LLM and AI compute** through a managed OpenAI-compatible inference endpoint and raw GPU leases, while allowing any containerized workload. Outcome: prepaid inference for `qwen3:0.6b` and a GPU container lease were validated on an RTX 4070 Laptop GPU provider.
4. **Retain a verifiable-task mode** for deterministic jobs, using an optimistic re-execution game on the same staking and reputation layer. Outcome: implemented with exact-hash comparison. Commit-reveal, randomized selection, and perceptual comparison remain future work.
5. **Deliver a live prototype and evaluate it.** Outcome: a Docker Compose stack, a Sepolia deployment, a simulator, a gas report, and a test suite of 42, 58, 54, 15, and 24 checks across five components (Chapter 9).

2.2 Motivation

2.2.1 Economic

Independent owners may have idle capacity, while requesters face limited or expensive supply. A marketplace can coordinate that supply without a custodial billing operator. Existing GPU marketplaces widen supply but still rely on platform servers for identity checks, listing verification, billing, and disputes [8].

2.2.2 Technical

Verifiable outsourced computation is an active research area. Interactive verification games [9], optimistic execution with challenge windows [10], and probabilistic spot checking [11] each address part of the problem for specific workloads. A practical marketplace must combine them with availability-based controls for work that cannot be re-executed. Building one forces explicit decisions about what the chain can know and what it cannot.

2.2.3 Educational

The project exercises the main topics of a blockchain course in one coherent system: ERC-20 tokens, escrow, access control, staking and slashing, on-chain state machines, event-driven off-chain services, reputation, dispute resolution, and mechanism design with adversarial participants.

2.2.4 Practical

Rental, inference, and deterministic task flows all produce observable outputs, which makes evaluation concrete. A lease produces heartbeat and settlement events. An inference call produces token counts and a payment. A deterministic job produces an image that the orchestrator stitches and pins.

2.3 Design Principles

Four principles guided decisions throughout the project and recur in later chapters.

- **The chain is the referee, not the worker.** Value movement and result validation live only in contracts.
- **Off-chain components are untrusted.** If an agent, gateway, or web page could steal funds or force a fake result, that is a defect.
- **State what is not proven.** A heartbeat shows that a key was live. A benchmark shows responsiveness and throughput. Neither proves the exact GPU model or the quality of an LLM answer. The report and the user interface say so.
- **Separate trust models for separate workloads.** Rentals use availability evidence. Deterministic jobs use re-execution. Both share one staking and reputation layer.

CHAPTER 3: LITERATURE SURVEY

The survey uses primary papers where they exist and official project documentation for systems whose operation has changed since their original publication. All online sources were accessed in October 2026. It extends the Review 1 survey [7] and groups prior work by the problem it addresses.

3.1 Programmable Settlement

Bitcoin showed that strangers can transfer value without a trusted intermediary [1]. Ethereum added a general-purpose virtual machine, so that escrow, tokens, and staking can be written as contracts [2], [12]. On-chain computation is expensive, so every compute marketplace built on Ethereum must execute work elsewhere and bring back only commitments and payments. The ERC-20 standard [4] and the audited OpenZeppelin library [13] supply the token and access-control primitives that ShaderMesh composes.

3.2 Decentralized Compute Marketplaces

Golem was an early Ethereum compute market and began with Blender rendering. Its documentation describes a peer-to-peer market of requestors and providers [14]. Result trust relies largely on redundancy and reputation rather than a precise on-chain referee.

Akash matches container deployments to providers through a reverse auction on a Cosmos-based chain [15]. It concentrates on availability and deployment, and does not verify the correctness of results.

Render Network focuses on GPU rendering for 3D content [16]. It is specialized to rendering and depends on reputation and review for quality.

iExec builds a Proof-of-Contribution protocol and relies on trusted execution environments to attest computation [17]. This gives stronger confidentiality guarantees but ties the design to specialized processor hardware.

Vast.ai is a centralized marketplace for rentable GPUs. The platform operator performs listing, billing, and dispute functions [8]. It is the closest commercial analogue of the supply side that ShaderMesh decentralizes.

3.3 Verification of Off-Chain Computation

TrueBit introduced an interactive verification game in which a challenger disputes a result and an on-chain referee resolves the dispute by bisection [9]. It is general, but the multi-round dispute logic is costly to implement and to run.

Arbitrum popularized optimistic execution: assume honesty, open a challenge window, and resolve on chain only if contested [10]. ShaderMesh borrows the optimistic pattern and the challenge window for deterministic tasks.

Livepeer verifies video transcoding by probabilistic spot checks backed by stake [11]. The idea that a fraction of work is re-executed, with penalties large enough to deter cheating, is the basis of the ShaderMesh task market and of its break-even analysis in Chapter 6.

Filecoin uses proofs of replication and spacetime to make stored data verifiable [18]. It solves verifiable storage rather than computation, and serves as a design analogue for recurring, seeded challenges against a provider.

Bittensor scores machine learning contributions by peer ranking [19]. The verification is subjective and specific to learning tasks, which makes it unsuitable as a source of deterministic penalties.

3.4 LLM Serving

Efficient LLM serving is an engineering field in its own right. Paged attention and similar techniques raise throughput on a given GPU [20]. ShaderMesh does not compete here. It treats the serving runtime as a replaceable backend behind an OpenAI-compatible interface [3], with an Ollama backend [21] used for the live validation.

3.5 Comparison with ShaderMesh

Table 3.1 compares the surveyed systems with ShaderMesh on workload, settlement, and verification approach. The comparison is limited to what each project documents. It does not rank the systems.

TABLE 3.1: Comparison of related systems with ShaderMesh

System	Workload	Settlement and trust	Verification approach	Gap addressed by ShaderMesh
Golem [14]	General compute, rendering	Ethereum token, peer-to-peer market	Redundancy and reputation	No separate availability model and no on-chain SLA penalty.
iExec [17]	Off-chain compute with confidentiality	Token and Proof-of-Contribution	Trusted execution environments	Depends on specialized hardware, not commodity GPUs.
Akash [15]	Container deployments	Reverse auction on a Cosmos chain	Availability and deployment focus	No verification of result correctness or hardware claims.
Render [16]	GPU 3D rendering	Token payment to node operators	Reputation and review	Rendering only.
Livepeer [11]	Video transcoding	Stake and probabilistic payment	Probabilistic spot checks	Domain specific. ShaderMesh generalizes the spot-check idea.
Bittensor [19]	Machine learning contributions	Token rewards	Peer scoring	Subjective scoring, no deterministic penalties.
TrueBit [9]	Verifiable off-chain computation	On-chain referee	Interactive verification game	Multi-round disputes are costly. Not aimed at rentals.
ShaderMesh	GPU and CPU rental, LLM inference, deterministic tasks	Streamed escrow, stake, bonds, reputation	Heartbeats and benchmark challenges for rentals. Optimistic re-execution for deterministic tasks.	Two explicit trust models over one staking layer. Limits are stated.

3.6 Synthesis

Prior work establishes the feasibility of programmable settlement, decentralized resource markets, probabilistic verification, hardware attestation, and optimized inference. The contribution of ShaderMesh is their integration under two explicit trust models. Non-replayable rentals use streamed escrow and availability-oriented controls. Deterministic jobs use optimistic verification. Both use a common staking and reputation layer.

The design does not remove all trust. Heartbeats show that a provider key was live, not that the service was good. Hardware attestation depends on the attester. Sampled checking depends on independent challengers and a suitable comparison rule. These limits are stated in Chapter 11 and are not hidden behind a general claim of verifiable AI.

CHAPTER 4: EXISTING SYSTEM AND IDENTIFIED GAP

4.1 Current Methods

The dominant method is centralized cloud rental. A customer selects an instance from a cloud provider or a GPU marketplace, accepts the operator account and billing rules, and reaches the machine through an API, SSH, a notebook, or a managed service. Community GPU marketplaces widen supply [8], but platform servers still perform identity checks, listing verification, billing, and dispute handling.

Decentralized compute systems take different approaches. Some match container deployments through a blockchain market [15]. Some specialize in rendering or video [16], [11]. Others verify deterministic execution through replication, challenge games, or trusted hardware [9], [17]. Each demonstrates useful components, but they make different trade-offs between generality, cost, and trust.

4.2 Problems and Limitations

- A centralized operator can change prices, suspend accounts, or control dispute outcomes.
- Billing and reliability histories may sit in a private database that participants cannot audit.
- Replicating every workload is expensive and does not suit interactive services such as inference endpoints.
- Hardware attestation can introduce a separate trusted authority or require specialized processors.
- Exact output comparison fails for many GPU and LLM workloads, where outputs differ across hardware or are subjective.
- Public provider machines create security risks for renter data and for the provider host.
- A blockchain cannot observe off-chain availability without signed reports, probes, or a trusted measurement service.

4.3 Identified Gap

The gap is a transparent compute-rental agreement that supports streamed escrow and service-level penalties, while keeping a distinct verification path for workloads that are actually checkable. Funds and deterministic rules must stay on chain, compute must stay off chain, and the design must not claim that one verification mechanism applies to every workload.

Fig. 4.1 places ShaderMesh between its two market participants and the external systems it depends on. Each arrow in the figure states the relationship and the protocol it uses.

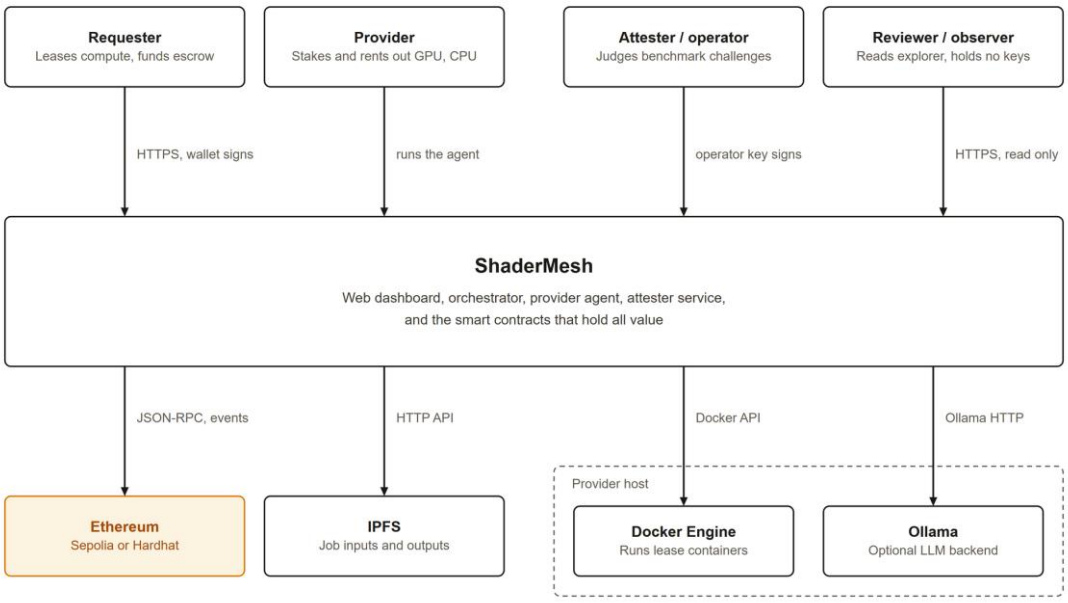


Fig. 4.1: System context: actors, ShaderMesh, and the external systems it depends on

4.4 Proposed Approach

ShaderMesh addresses the gap in four steps. First, it escrows renter funds in a contract and releases them one interval at a time against provider heartbeats. Second, it requires providers to stake and bonds part of that stake for the duration of a lease, so that missed service has a price. Third, it gates market entry on an attestation and keeps testing attested providers with seeded benchmark challenges. Fourth, it keeps a separate optimistic re-execution market for deterministic jobs, so that verification is applied only where it is possible.

The approach leaves some trust in place. The attester judges benchmark answers, the requester reports inference usage, and heartbeat evidence comes from the provider itself. Chapter 11 returns to each of these limits.

CHAPTER 5: REQUIREMENTS

The authoritative requirements are in the project specification [22]. This chapter summarizes them and reports the implementation status that was measured against the repository, not against roadmap intent.

5.1 Actors

The specification names seven actors. The **requester** rents compute, buys inference, or posts a deterministic job. The **provider** owns capacity, stakes, and serves leases and models. A **challenger** is a provider that re-executes tasks to verify them. The **attester** is a trusted service that records hardware attestations and judges benchmark challenges. The **operator** runs a local or testnet deployment. A **reviewer** evaluates the project. Off-chain components act on behalf of these actors but hold no funds.

5.2 Functional Requirements

There are 75 functional requirements in ten modules, with identifiers such as FR-RENT-04. Each is marked must or should. Table 5.1 lists the modules, the requirement counts, and the main behavior each module must provide.

TABLE 5.1: Functional requirement groups

Module	IDs	Count	Main behavior required
Token	FR-TKN	3	ERC-20 payment token, owner mint, rate-limited faucet.
Provider registry	FR-REG	10	Register, stake and unstake free stake, reputation EMA, restricted bond and slash, eligibility, attestation, benchmark challenges.
Job market	FR-JOB	9	Post jobs with escrow, claim with bond, submit commitments, optimistic settlement, timeout reclaim.
Verification	FR-VER	8	Challenge, dispute, vote, majority resolution, duplicate-vote prevention. Commit-reveal and perceptual policy are should-level.
Orchestrator	FR-ORC	6	Event index in Postgres, split, stitch, read API, live channel, no custody.
Provider agent	FR-AGT	10	Auto register and stake, claim and execute tasks, sandboxing, challenger mode, serve leases and inference, answer challenges.
Web	FR-WEB	8	Wallet connect, rent, models and playground, provider console, explorer, wallet-only writes.
Simulator	FR-SIM	4	Break-even spot-check rate, trade-off analysis, mixed populations, rental economics.
Rental market	FR-RENT	10	Listings, manifest-bound requests, bond on acceptance, streamed escrow, strict heartbeats, termination and slashing, extension.
LLM inference	FR-LLM	7	Model offers, OpenAI-compatible server, gateway, cumulative usage settlement, signed request authorization.

5.2.1 Rental market

The rental module is the primary market. A provider lists capacity with a price per interval, an interval length, and minimum and maximum durations (FR-RENT-01). A requester opens a lease by escrowing the full price for the requested duration and binding a content-addressed workload manifest to a pending request (FR-RENT-02, FR-RENT-10). The lease clock and the provider bond start only when the provider accepts. During the lease, a provider heartbeat must back every settled interval (FR-RENT-05). Anyone can terminate a lease after a missed heartbeat plus a grace period, which slashes the bond and refunds the unused escrow (FR-RENT-06).

5.2.2 Inference

A provider advertises a model and a price per token (FR-LLM-01). A requester opens a prepaid credit, signs each request with a short-lived EIP-191 signature [23],

and submits monotonic cumulative usage to settle against the credit (FR-LLM-04, FR-LLM-07). The gateway routes only to the provider fixed by the credit.

5.2.3 Verifiable tasks

A requester posts a job with an input content identifier (CID), a task count, and a reward per task, and the full reward is escrowed (FR-JOB-01, FR-JOB-02). A provider claims a task with a bond, submits a hash commitment and an output CID, and is paid after a challenge window unless a dispute is upheld (FR-JOB-03 to FR-JOB-05). Any eligible provider other than the task owner may challenge by submitting an independently computed hash (FR-VER-01).

5.2.4 Provider operations and attestation

A provider registers, stakes, and obtains a fresh attestation before it can list capacity or offer a model. The registry accepts benchmark challenges against attested providers. A failed or missed challenge slashes free stake and lowers reputation (FR-REG-09, FR-AGT-10).

5.3 Non-Functional Requirements

Twelve non-functional requirements constrain security, performance, reliability, usability, portability, maintainability, and cost. They are listed in Table 5.2.

TABLE 5.2: Non-functional requirements

ID	Requirement
NFR-SEC-01	Off-chain components must not custody funds or force result acceptance.
NFR-SEC-02	Contracts follow checks-effects-interactions and guard token movements against reentrancy.
NFR-SEC-03	No secrets in the repository. Only example environment files with test values.
NFR-SEC-04	Provider job execution is sandboxed with CPU, memory, and network limits.
NFR-PERF-01	Core on-chain operations stay within a modest gas budget suitable for a testnet demonstration.
NFR-PERF-02	The dashboard reflects a new on-chain event within a few seconds.
NFR-REL-01	The local stack starts reliably with one command and correct boot ordering.
NFR-REL-02	The orchestrator index is reconstructable from chain history after data loss.
NFR-USE-01	The dashboard meets WCAG AA contrast and shows visible focus states [24].
NFR-PORT-01	The system runs on Windows, macOS, and Linux through Docker.
NFR-MAINT-01	Code passes linting and type checks. Contract logic is covered by tests.

ID	Requirement
NFR-COST-01	The demonstration runs at zero cost on a local devnet and at low cost on a testnet.

5.4 Traceability and Status

Status was assessed from behavior in the repository. Of the 75 functional requirements, 64 are done, 6 are partial, and 5 are planned. Table 5.3 groups requirements and gives the evidence or the gap. Planned items are all should-level.

TABLE 5.3: Requirements traceability and implementation status

Requirement	Status	Evidence or gap
FR-TKN-01 to 03	Done	SmeshToken: ERC-20, owner mint, rate-limited faucet.
FR-REG-01 to 07, 09, 10	Done	ProviderRegistry and contract tests: registration, stake, bond, eligibility, availability, EMA outcomes, benchmark challenges.
FR-REG-08	Partial	Free-form capability JSON and attested GPU model and VRAM exist. CPU and region are not typed registry fields.
FR-JOB-01 to 09	Done	JobMarket, agent integration tests, orchestrator projection.
FR-VER-01 to 05	Done	Verification: eligible non-self voting, disputes, quorum, majority resolution, duplicate-vote prevention.
FR-VER-06 to 08	Planned	Commit-reveal, reputation-weighted random selection, and perceptual verification are not implemented.
FR-ORC-01 to 06	Done	Deployment discovery, Postgres projection and replay, split, stitch, API, SSE, no custody.
FR-AGT-01 to 07, 09, 10	Done	Worker, challenger, IPFS, sandbox, retries, inference endpoint, benchmark responder.
FR-AGT-08	Partial	Serves accepted leases with strict heartbeats, but runs provider-configured workloads, not requester-defined containers.
FR-WEB-01 to 03, 05 to 08	Done	Wallet, rental, prepaid inference, provider console, explorer, copy-safe values, wallet-only writes.
FR-WEB-04	Planned	Requester job posting and live tile assembly are not implemented. The explorer shows jobs and tasks.
FR-SIM-01 to 04	Done	Simulator covers verification and rental economics with 15 tests.
FR-RENT-01 to 07, 09, 10	Done	RentalMarket with agent and web clients.
FR-RENT-08	Partial	Lease success or failure updates the EMA. No proportional uptime score or lease dispute mechanism.
FR-LLM-02, 04, 07	Done	Provider endpoint, signed authorization, cumulative settlement, refunds.
FR-LLM-01, 03, 06	Partial	Context length is not an offer field. The gateway does not probe health or fail over. Lease workloads are provider-configured.
FR-LLM-05	Planned	No sampled inference reputation mechanism.
NFR-SEC-01 to 04	Done (prototype)	Custody boundaries, OpenZeppelin guards, secret templates, gateway controls, constrained workers. Docker socket authority remains a host risk.
NFR-REL-01, 02	Done	Compose health ordering. Snapshot plus ordered replay.
NFR-MAINT-01	Done	CI runs compile, tests, lint, type checks, build, gas, smoke, and browser checks. No property-based fuzzing.

Requirement	Status	Evidence or gap
NFR-PERF-01, 02, USE-01, PORT-01, COST-01	Partial	Gas report and responsive browser checks exist. Formal accessibility, latency, cross-platform, and public cost acceptance are not complete.

Section 12 of the specification gives the same table in full. The partial and planned items are carried into the limitations in Chapter 11 and the future work in Chapter 12.

CHAPTER 6: SYSTEM DESIGN

6.1 Architecture Overview

ShaderMesh is a set of cooperating components arranged around a trust boundary. Fig. 6.1 shows the container view. The right-hand block holds the six contracts, which are the only place where value moves. The left-hand block holds the off-chain components: the web application, the orchestrator with its Postgres projection and IPFS client, the provider agent with its Docker and Ollama dependencies, and the attester service. Every arrow that crosses the boundary is a JSON-RPC call signed by a key that belongs to a user, a provider, or the attester.

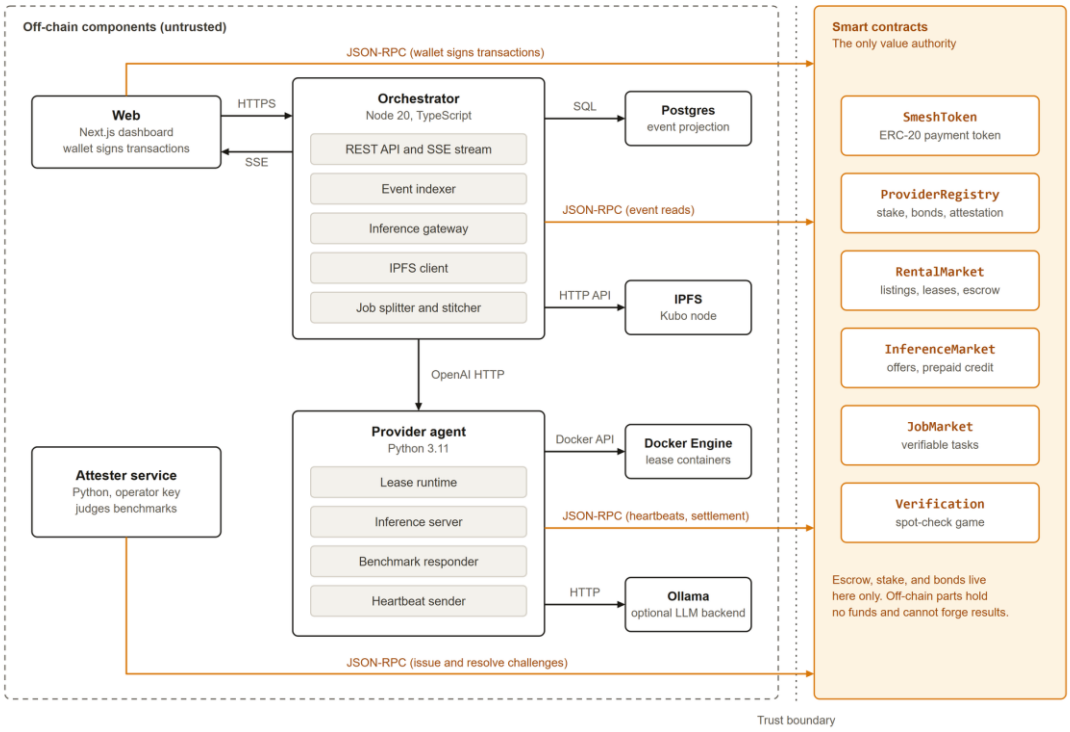


Fig. 6.1: Container view: off-chain components (untrusted) and the contracts that hold all value

The components divide the work as follows.

- **Contracts** hold funds, stake, bonds, lifecycle state, commitments, and reputation. They enforce every rule that involves value.
- **Provider agent** advertises capacity, accepts or rejects lease requests, runs accepted workloads in sandboxed containers, serves inference, sends heartbeats, executes deterministic tasks, and answers benchmark challenges.

- **Attester service** records hardware attestations and issues, judges, and expires benchmark challenges with the registry attester key.
- **Orchestrator** indexes chain events into Postgres, serves a read API and a live event stream, routes authorized inference requests, and splits and stitches deterministic jobs through IPFS.
- **Web application** reads indexed state from the orchestrator and builds every transaction for the user wallet to sign.
- **IPFS** stores manifests, inputs, and outputs by content identifier. **Postgres** stores a derived cache and replay keys.

6.2 Trust Boundary

The rule that shapes the design is that no off-chain component can steal funds or force the acceptance of a result. Fig. 6.2 summarizes what each component can and cannot do. It also marks the attester as a separate, operator-trusted role: it can judge benchmark answers, but only within bounds that the registry enforces.

	Component	Can	Cannot
On-chain value authority	**Smart contracts** Hold all value	Hold escrow, stake, and bonds Pay providers, refund requesters Slash bonds and free stake	Run workloads or see their output Move funds outside the coded rules Act without a signed transaction
Operator trusted to judge	**Attester service** Operator key	Issue benchmark challenges Judge an answer as pass or fail	Move or custody user funds Slash bonded stake, or beyond a fixed amount Penalise by silence: unjudged lapses
Off-chain untrusted	**Orchestrator** Untrusted	Index chain events into Postgres Route metered inference to providers Split, index, and stitch job output	Custody funds or sign for users Change balances or payouts Forge a result the chain accepts
	Provider agent Untrusted	Run containers and serve inference Send heartbeats, answer challenges Accept or reject lease requests	Be paid without heartbeat evidence Pass a benchmark it did not compute Hide downtime past the grace period
	Web Untrusted	Build transactions for wallet signing Display chain and orchestrator data	Hold keys or sign for a user Change state shown by the contracts

Fig. 6.2: What each component can and cannot do

The worst case for a misbehaving off-chain component is degraded service. A faulty orchestrator can return stale data or refuse to route a request, but it cannot change a balance. A malicious provider agent can fail to serve, but it cannot be paid without heartbeat evidence or pass a benchmark it did not compute. A compromised web

page can ask the user to sign a bad transaction, which is a wallet-level risk common to all decentralized applications, but it cannot sign for the user.

6.3 On-Chain and Off-Chain Boundary

Placing each piece of state on the correct side of the boundary keeps gas costs low and keeps trust assumptions explicit. Table 6.1 lists what the chain stores and what stays off chain.

TABLE 6.1: Placement of data and logic across the trust boundary

Item	Location	Reason
Escrow, stake, bonds, payouts, refunds	On chain	Value must be controlled by contract rules.
Lease, task, credit, and challenge state	On chain	State machines are the enforcement mechanism.
Result commitment (keccak256 hash)	On chain	Compact, comparable, and tamper evident.
Reputation score and outcome counts	On chain	Public record that no database controls.
Workload manifest, job input, job output	IPFS, CID on chain	Large payloads do not belong in contract storage.
Container execution, model inference	Provider machine	Heavy compute cannot run on chain.
Event index and API views	Postgres, rebuildable	Fast queries. Not a source of truth.
Inference request replay keys	Postgres, durable	Operational security state, not chain state.
Hardware attestation record	On chain, written by attester	Gate for listings and offers.
Benchmark answer judgment	Attester, outcome on chain	Needs off-chain recomputation. Trusted role.

6.4 Data Model

The data model is the set of structures that the contracts hold and the orchestrator projects. Table 6.2 lists the main entities and their key fields.

TABLE 6.2: Main entities and key fields

Entity	Key fields
Provider	registered, active, stake, bonded, reputation (0 to 10,000), completed, failed, capabilities (free-form JSON).
Attestation	attestedAt, GPU model, VRAM in GB, benchmark score. Fresh for a time-to-live (7 days by default). Cleared by a failed challenge.
Benchmark challenge	id, provider, seed, issuedAt, deadline, respondedAt, status (open, responded, passed, failed, lapsed), score, outputHash.
Listing	provider, price per interval, interval seconds, minimum and maximum intervals, availability, active lease, resource specification.
Lease	listing, requester, provider, manifest, price, funded, heartbeated and settled intervals, escrow, bond, start, status.
Inference offer	provider, model identifier, price per token, active.
Inference credit	offer, requester, provider, snapshotted price per token, escrow, paid, cumulative prompt and completion tokens, active.
Job and task	Job: requester, input CID, task count, completed tasks, reward per task, verification policy. Task: provider, result hash, output CID, status, bond, claimed and submitted times.
Dispute	opened, resolved, provider hash, deadline, agree and disagree counts, first disagreeing challenger.

6.5 Token Economics

SMESH is an ERC-20 utility token [4] used for three purposes: payment from requesters to providers, collateral that providers stake, and rewards to honest challengers. On test networks a rate-limited faucet mints 1,000 SMESH per address per day. It has no value outside the test environment.

6.5.1 Stake, bond, and slash

A provider must register and stake before it is eligible. Stake is a security deposit. When a provider takes on work, part of the stake is **bonded** to that work and cannot be withdrawn. Available stake is the total stake minus the bonded amount. A **slash** confiscates stake and sends it to a beneficiary: the requester for a missed heartbeat or an abandoned task, the first correct challenger for upheld cheating, and the treasury for a failed benchmark. Fig. 6.3 shows the flows between requester, provider, markets, and treasury.

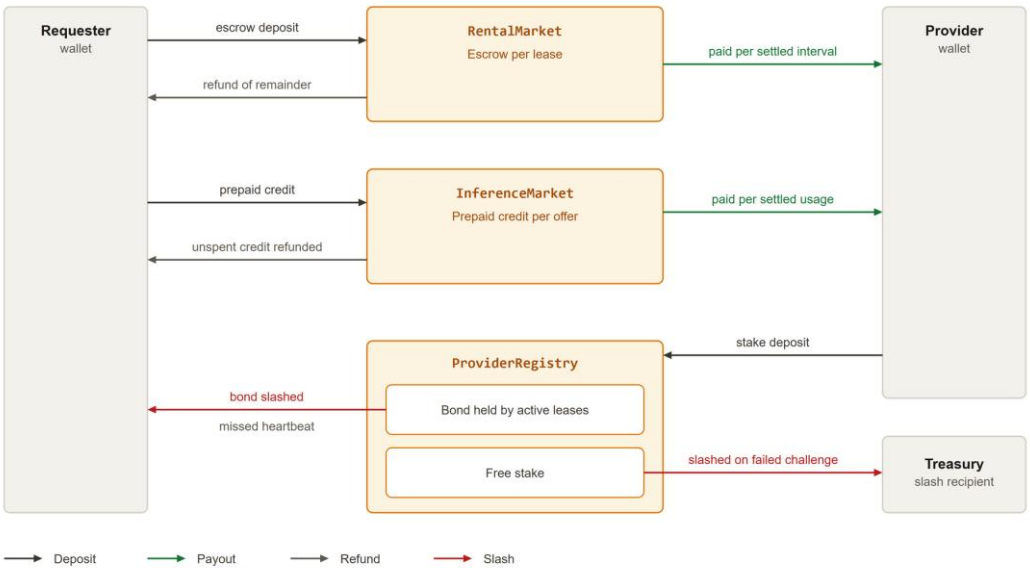


Fig. 6.3: Token flows: deposits, per-interval payouts, refunds, and slashes

6.5.2 Reputation

Reputation is an integer from 0 to 10,000, starting at 5,000. After each outcome it moves toward 10,000 on success or toward 0 on failure by an exponential moving average with smoothing constant K = 5:

$$reputation' = (4 \times reputation + target) / 5, \quad target = 10{,}000 \text{ on success or } 0 \text{ on failure}$$

6.5.3 Protocol parameters

Table 6.3 lists the default parameters. Each is set by the contract owner and can be changed without redeploying the contract.

TABLE 6.3: Default protocol parameters

Parameter	Default	Contract
Minimum stake for eligibility	50 SMESH (deployment setting)	ProviderRegistry
Attestation time-to-live	7 days	ProviderRegistry
Challenge window	15 minutes (120 seconds in the local stack)	ProviderRegistry
Challenge slash	One fifth of minimum stake, 10 SMESH	ProviderRegistry
Bond per lease	50 SMESH	RentalMarket
Heartbeat grace period	5 minutes	RentalMarket
Bond per task	10 SMESH	JobMarket
Task challenge window	1 hour	JobMarket
Task claim timeout	1 hour	JobMarket

Parameter	Default	Contract
Vote window	30 minutes	Verification
Quorum	1	Verification
Faucet amount and cooldown	1,000 SMESH per 1 day	SmeshToken

6.5.4 Deterrence for verifiable tasks

Let R be the reward per task, B the bond, and s the probability that a task is spot checked by at least one honest challenger. A provider that skips the computation keeps R if it is not checked and loses B if it is. The expected value of cheating per task is:

$$EV(cheat) = (1 - s)\,R - s\,B$$

Cheating is deterred when this value is not positive, which gives the break-even spot-check rate:

$$s^* = R / (R + B)$$

A larger bond lowers the rate needed for deterrence and therefore lowers verification cost. With R = B = 100, the break-even rate is 0.5. The simulator in Chapter 10 reproduces this value.

6.5.5 Deterrence for rentals

A rented result cannot be re-executed, so the corresponding quantity concerns availability. Let V be the value a provider captures per interval by claiming service it did not deliver, D the bond at risk, and h the probability that a false claim is detected. Deterrence needs h D to exceed (1 - h) V, which gives:

$$EV(\text{false heartbeat}) = (1 - h)\,V - h\,D, \quad h^* = V / (V + D)$$

With V equal to one interval price of 10 and D equal to a bond of 500, h* is about 0.020. Even weak detection deters false claims when the bond is large relative to a single interval of income.

6.6 State Machines

The rental lease has six states. Fig. 6.4 shows the transitions and the effect on escrow and bond in each terminal state. A request escrows the full price while the lease is pending. Acceptance starts the clock and bonds the provider. A lease ends normally when the requester ends it or when every funded interval has heartbeat evidence. It is

terminated when a heartbeat is missed beyond the grace period, which slashes the bond to the requester.

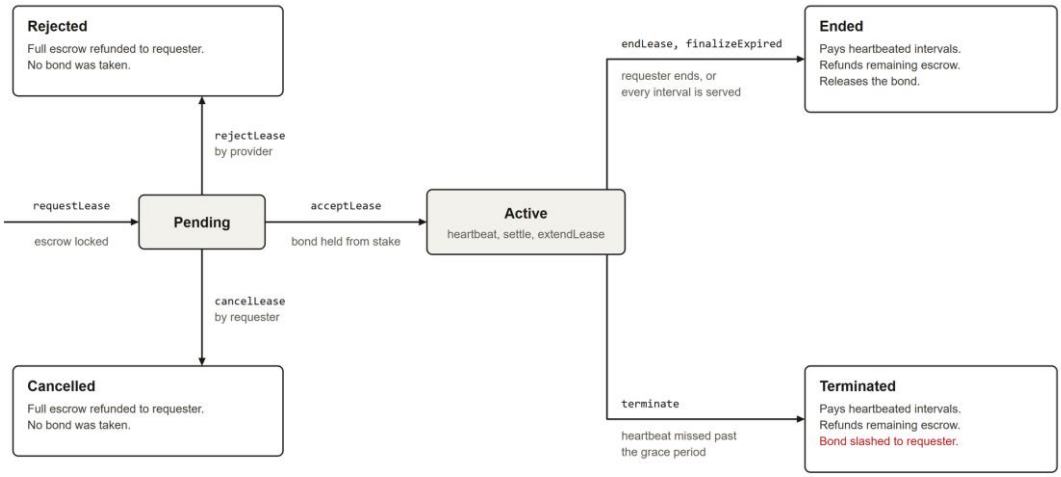


Fig. 6.4: Lease states and the treatment of escrow and bond at each terminal state

A verifiable task moves through the states in Table 6.4. The contract reserves a sixth state, Failed, which is not used in the current implementation.

TABLE 6.4: Verifiable task states and transitions

State	Entered by	Next states
Open	Job posted, or task reopened after a slash	Claimed
Claimed	claimTask by an eligible provider, bond locked	Submitted, or Open (timeout reclaim with bond slashed to requester)
Submitted	submitResult with a hash and output CID	Paid (finalizeTask after the window), Disputed (first challenge)
Disputed	First challenge in Verification	Paid (provider vindicated), or Open (cheating upheld, bond slashed to challenger)
Paid	Settlement	Terminal. Reward paid, bond released, reputation raised.

The benchmark challenge has its own state machine, which is described with the registry in Chapter 7.

CHAPTER 7: SMART CONTRACT DESIGN

The contracts are written in Solidity 0.8.24 and build on OpenZeppelin [13]: ERC20, Ownable, SafeERC20, and ReentrancyGuard. Every function that moves tokens either follows the checks-effects-interactions order or is guarded against reentrancy, or both. The sections below describe each contract by state, key functions, events, access control, and invariants. Function lists give the main entry points, not every view.

7.1 SmeshToken

SmeshToken is an ERC-20 token named ShaderMesh Token with symbol SMESH. It adds an owner-controlled `mint` and a `faucet` that mints 1,000 SMESH to the caller. A per-address cooldown (1 day by default, owner adjustable) limits the faucet. The faucet exists only for test networks. The `Faucet` event records each claim.

7.2 ProviderRegistry

ProviderRegistry is the shared staking, reputation, and attestation layer. All market contracts read from it and the authorized ones write to it.

7.2.1 State

Each provider has a record of registration and active flags, total stake, bonded amount, reputation, completed and failed counters, and a capability string. A separate attestation record holds the attestation time, VRAM, benchmark score, and GPU model. The registry also holds the attester address, the attestation time-to-live, the challenge table, an open-challenge index per provider, the challenge window, the challenge slash amount, and the treasury address.

7.2.2 Key functions

Table 7.1 lists the registry entry points with the caller allowed to use each.

TABLE 7.1: ProviderRegistry functions

Function	Caller	Effect
register, updateCapabilities, setActive	Provider	Create the record with reputation 5,000, change capabilities, toggle availability.
stake, unstake	Provider	Deposit SMESH. Withdraw only free stake, and only with no open challenge.
isEligible, availableStake, reputationOf	Anyone (view)	Eligibility needs registered, active, and stake at or above the minimum.
bond, unbond, slash, recordOutcome	Authorized contracts	Lock and release stake, confiscate stake to a beneficiary, update reputation.
attest, isAttested	Attester, anyone (view)	Record GPU, VRAM, and score. Fresh within the time-to-live.
issueChallenge	Attester	Open a seeded challenge for an attested provider with no open challenge.
respondChallenge	Challenged provider	Submit output hash and score before the deadline.
resolveChallenge	Attester	Pass or fail an answered challenge within one further window.
expireChallenge	Anyone	Fail a missed deadline, or lapse an answer that was never judged.
setAuthorized, setMinStake, setAttester, setAttestationTtl, setTreasury, setChallengeParams	Owner	Configuration.

7.2.3 Events and access control

The registry emits events for registration, capability updates, stake, unstake, bond, unbond, slash, reputation changes, availability, authorization, attester and treasury changes, attestation and revocation, and the three challenge phases (`ChallengeIssued`, `ChallengeResponded`, `ChallengeResolved`). Three roles exist. The **owner** configures parameters and authorizes contracts. **Authorized contracts** (the markets and Verification) alone can bond, unbond, slash, and record outcomes. The **attester** alone can attest, issue challenges, and judge answers.

7.2.4 Invariants

- Bonded stake never exceeds total stake. A slash reduces the bonded amount first. A challenge slash takes free stake only.
- Only free stake can be withdrawn, and not while a challenge is open.
- A provider has at most one open challenge.
- Only authorized contracts change stake, bond, and reputation on behalf of a market.

7.3 RentalMarket

RentalMarket implements listings, manifest-bound lease requests, strict per-interval heartbeats, streamed settlement, and termination. It is the primary market.

7.3.1 State

A listing holds the provider, the price per interval, the interval length in seconds, the minimum and maximum number of intervals, an availability flag, the active lease identifier, and a resource specification string. A lease holds the listing, requester, provider, price, funded, heartbeated, and settled interval counts, escrow, bond, start time, status, and the manifest. A nested mapping records the heartbeat time of every interval of every lease.

7.3.2 Key functions

Table 7.2 lists the entry points of the market.

TABLE 7.2: RentalMarket functions

Function	Caller	Effect
createListing	Eligible, attested provider	Publish capacity with price, interval, and duration bounds.
setListingAvailable	Provider	Toggle a listing that has no active lease.
requestLease	Requester	Escrow price times intervals and bind a manifest. State Pending.
acceptLease	Provider	Start the clock, bond the provider, reserve the listing. State Active.
rejectLease, cancelLease	Provider, requester	Refund the full escrow from a pending request.
heartbeat	Provider	Record evidence for the current interval only, in order.
settle	Anyone	Pay all completed intervals in the heartbeated prefix.
extendLease	Requester	Add intervals and escrow, up to the maximum duration.
endLease	Requester	Pay served intervals, refund the rest, release the bond.
finalizeExpired	Anyone	Close a lease whose funded intervals all have heartbeat evidence.
terminate	Anyone	After the missed-heartbeat deadline, pay the served prefix, refund the rest, slash the bond to the requester.

Fig. 7.1 shows the lifecycle as a sequence of calls between requester, provider, and contract, including the two ways a lease can end.

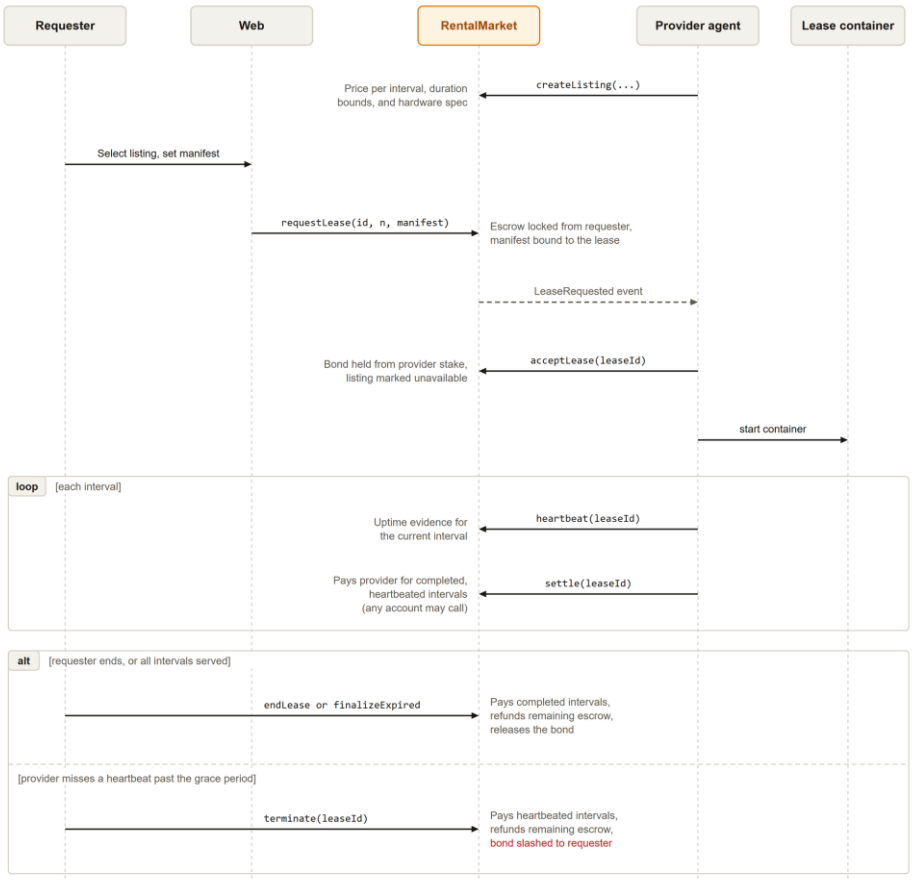


Fig. 7.1: Rental lifecycle: listing, lease, heartbeats, settlement, and the two ways a lease ends

7.3.3 Heartbeat rule

The heartbeat function computes the current interval as the elapsed time since the start divided by the interval length. It accepts a heartbeat only from the lease provider, only for the current interval, only once, and only if the interval equals the number of intervals already heartbeated. A provider therefore cannot cover a missed interval with a later heartbeat. The payable count is the minimum of completed intervals, funded intervals, and heartbeated intervals. The lease is down when the first unheartbeated interval has ended and the grace period has also passed.

7.3.4 Invariants

- Settled intervals never exceed heartbeated intervals, and heartbeated intervals never exceed funded intervals.
- Escrow decreases only by the amount paid for newly settled intervals, so funded escrow equals provider payment plus requester refund at every terminal state.

- The clock, bond, and listing reservation begin only on provider acceptance.
- A slash on termination takes the lease bond, which was set aside at acceptance.

7.4 InferenceMarket

InferenceMarket handles managed inference with prepaid credit. A provider creates an offer with a model identifier and a price per token. It must be eligible and attested. A requester opens a credit against an offer, which escrows funds and snapshots the offer price. After each response the requester calls `settleUsage` with the cumulative prompt and completion token counts. The contract pays the provider for the difference since the last settlement and rejects a payment above the remaining escrow or a counter that decreases. Replaying the same counters pays zero. The requester can close the credit and recover the remaining escrow.

Table 7.3 lists the entry points of the inference market.

TABLE 7.3: InferenceMarket functions

Function	Caller	Effect
createOffer, deactivateOffer	Eligible, attested provider	Publish or withdraw a model offer at a price per token.
openCredit	Requester	Escrow funds and snapshot the price. Offer provider must be eligible and attested.
settleUsage	Requester	Record cumulative usage and pay the delta to the provider from escrow.
closeCredit	Requester	Refund remaining escrow and deactivate the credit.

The invariants are that payment never exceeds escrow, counters never decrease, and only the requester can settle or close. The requester-metered design has a known limit: the contract prevents overdraw and decreasing counters but cannot prevent a requester from under-reporting usage. Chapter 11 discusses it.

7.5 JobMarket

JobMarket is the escrow and state machine for verifiable tasks. A requester posts a job with an input CID, a task count, a reward per task, and a verification policy of 0 (exact keccak256) or 1 (perceptual, off-chain assisted). The contract escrows reward times task count. Providers claim tasks, which bonds 10 SMESH each, submit a result hash and an output CID, and are paid by `finalizeTask` after the challenge window if no dispute is upheld. Anyone can call `reclaimTimedOut` on a task that was claimed but not submitted within the claim timeout, which slashes the bond to the

requester and reopens the task. Three functions are restricted to the Verification contract: `markDisputed`, `resolveCheating`, and `resolveHonest`.

Table 7.4 lists the entry points of the job market.

TABLE 7.4: JobMarket functions

Function	Caller	Effect
postJob	Requester	Escrow reward times task count. Emit JobPosted.
claimTask	Eligible provider	Bond stake and move the task to Claimed.
submitResult	Task provider	Commit result hash and output CID. State Submitted.
finalizeTask	Anyone	After the challenge window, pay the reward and release the bond.
reclaimTimedOut	Anyone	Slash an abandoning provider and reopen the task.
markDisputed, resolveCheating, resolveHonest	Verification only	Freeze, slash and reopen, or pay a disputed task.

7.6 Verification

Verification is the dispute game for task results. An eligible provider other than the task owner calls `challenge` with an independently computed hash during the task challenge window. The first call opens a dispute, copies the owner commitment, starts a vote window of 30 minutes, and freezes payout through `markDisputed`. Each later challenger adds one vote. A vote agrees when its hash equals the owner hash and disagrees otherwise. The contract prevents a second vote by the same address and prevents self-challenge. After the vote window, anyone can call `resolve`. Cheating is upheld when disagreeing votes exceed agreeing votes and reach the quorum. In that case the first disagreeing challenger receives the slashed bond and a reputation success. Otherwise the owner is paid and the first disagreeing challenger takes a reputation failure.

The current design has known simplifications. Votes are plaintext, so a challenger can copy the owner commitment. Challengers volunteer instead of being selected at random, and votes are not weighted by reputation. The default quorum of one makes a single honest challenger decisive. These are listed as planned hardening in Chapter 12.

7.7 Contract Function Summary

Table 7.5 summarizes the surface of the six contracts and the access control that applies to state-changing functions.

TABLE 7.5: Summary of contract surface and access control

Contract	State-changing functions	Restricted by	Reentrancy guard
SmeshToken	mint, faucet, setFaucetCooldown	Owner (mint, cooldown)	Not needed
ProviderRegistry	register, stake, unstake, bond, unbond, slash, recordOutcome, attest, issueChallenge, respondChallenge, resolveChallenge, expireChallenge	Owner, authorized contracts, attester, and the challenged provider	Uses SafeERC20 transfers
RentalMarket	createListing, requestLease, acceptLease, rejectLease, cancelLease, heartbeat, extendLease, settle, endLease, finalizeExpired, terminate	Provider or requester per function. Settle, finalize, and terminate are open	Yes, on token movements
InferenceMarket	createOffer, deactivateOffer, openCredit, settleUsage, closeCredit	Provider or requester per function	Yes, on token movements
JobMarket	postJob, claimTask, submitResult, finalizeTask, reclaimTimedOut, markDisputed, resolveCheating, resolveHonest	Verification contract for dispute outcomes. Owner for parameters	Yes, on token movements
Verification	challenge, resolve	Eligible non-owner provider. Resolve is open after the window	Calls guarded markets

7.8 Benchmark Challenge Mechanism

Attestation is a record written once. A provider could pass it with real hardware and then stop delivering that performance. Benchmark challenges close this gap by testing attested providers repeatedly, with work that cannot be prepared in advance.

7.8.1 Protocol

1. The attester calls `issueChallenge` for an attested provider that has no open challenge. The seed is keccak256 of `block.prevrandao`, the provider address, and the challenge identifier, so the provider cannot precompute the answer before issuance. The deadline is the issue time plus the challenge window.
2. The provider runs a chain of 500,000 SHA-256 [25] iterations starting from the seed, measures a score, and calls `respondChallenge` with the output hash and the score. The score is tokens per second multiplied by 100 on a fixed prompt when an Ollama model is configured, and otherwise the hash rate in kilohashes per second.

3. The attester recomputes the hash chain. The challenge passes when the hash matches and the score is at least the attested score multiplied by the minimum score ratio (50 percent by default). A pass refreshes the attestation and records a reputation success.
4. A wrong answer judged by the attester fails the challenge. A missed deadline is closed by `expireChallenge`, which any account may call.
5. A failed challenge slashes up to the challenge slash amount from **free stake only**, sends it to the treasury, revokes the attestation, and records a reputation failure. Active leases continue under their existing bonds.
6. An answered challenge that the attester never judges within one further window lapses with no penalty.

Fig. 7.2 shows the message flow and Fig. 7.3 shows the challenge states and the conditions for each transition.

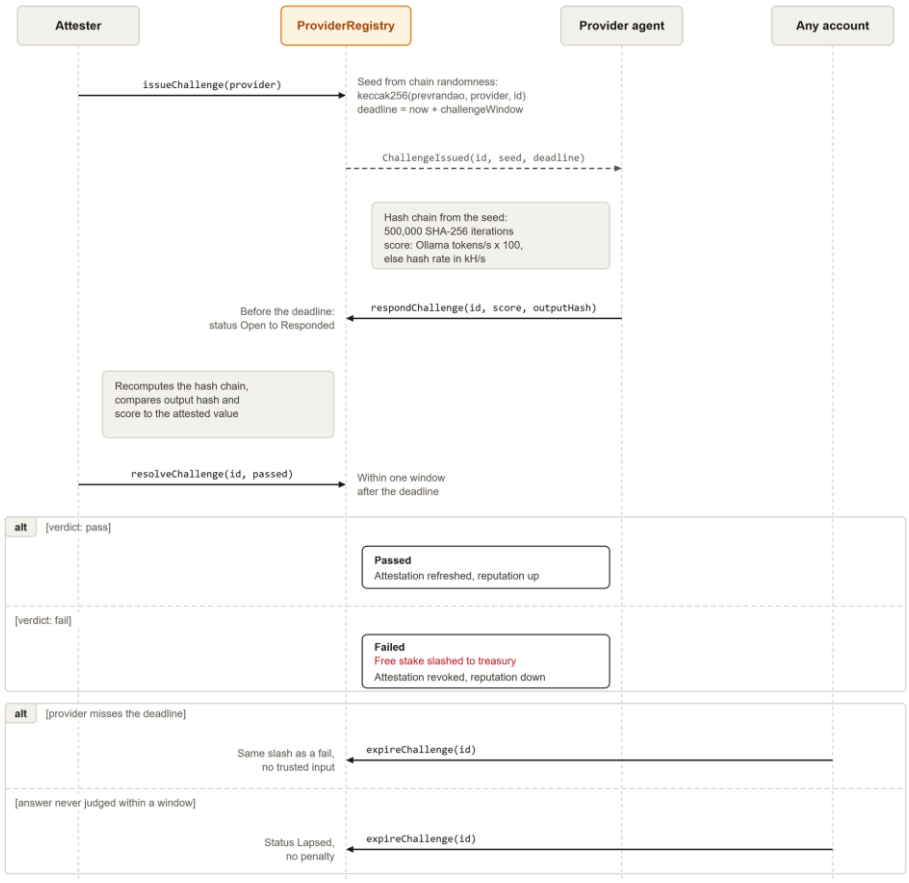


Fig. 7.2: Benchmark challenge: issue, answer, judge, and the permissionless expiry paths

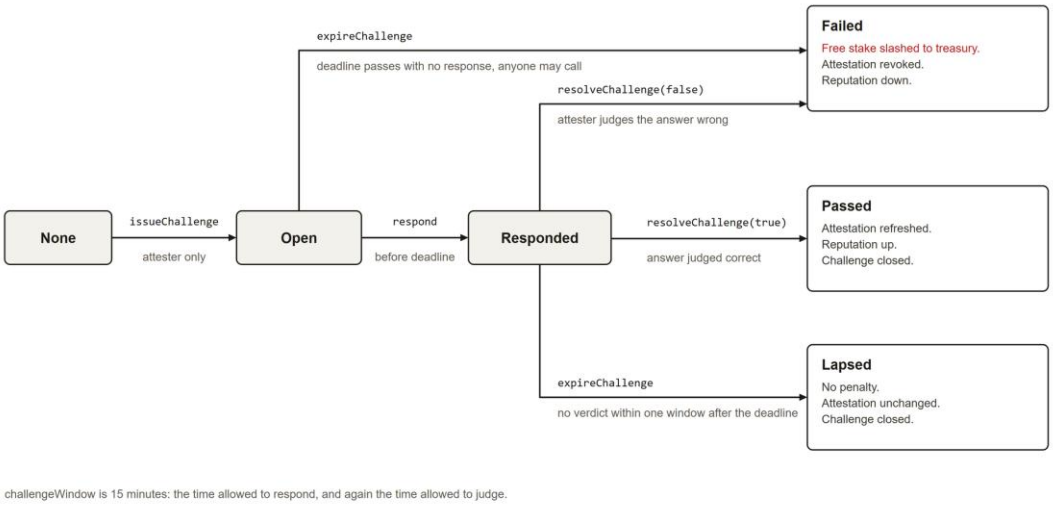


Fig. 7.3: Benchmark challenge states and transition conditions

7.8.2 Security reasoning

Each rule in the mechanism closes a specific attack or abuse path.

- **Free stake only.** Bonded stake backs the compensation owed to requesters of active leases and tasks. If a challenge could slash it, a failed benchmark would reduce the funds that protect a requester who is not part of the dispute, and the attester would gain a path to drain lease bonds. Restricting the slash to free stake keeps the invariant that bonded stake is whole. The slash is also capped by the free stake that exists, so it cannot underflow.
- **Unstake lock.** Without the lock, a provider that has just been issued a challenge could withdraw all free stake, then fail or ignore the challenge with nothing left to slash. The registry therefore rejects `unstake` while the provider has an open challenge.
- **Permissionless expiry.** A provider that stays silent cannot avoid a penalty, and the attester cannot shield a friendly provider by not acting. After the deadline anyone can call `expireChallenge`, and the outcome depends only on the clock and on the absence of a response, so it needs no trusted input.
- **Lapse path.** If the provider answered but the attester never judges, the challenge would stay open and would block unstaking indefinitely. After one further window, anyone can lapse it with no penalty. A silent or offline attester cannot hold a provider locked, and it cannot slash late.
- **Seeded work.** A seed that depends on `block.prevrandao`, the provider, and the identifier prevents precomputation and replay across providers. The value is not

a source of strong randomness against block proposers. Its purpose here is to prevent a provider from preparing an answer before the challenge exists.

- **One open challenge.** Limiting each provider to one open challenge bounds the load on the provider and the number of penalties that can be pending.

The mechanism leaves the attester as a trusted judge of answers. The registry bounds that trust. The attester can fail a correct answer, which costs the provider up to the slash amount and its attestation, but it cannot touch bonded stake, move funds to itself (the slash goes to the treasury), or exceed the configured amount per challenge. A failed provider must be attested again before another challenge can be issued, and re-attestation is also an attester action. The attester should therefore be run by a party that the participants accept, and its decentralization is future work.

CHAPTER 8: IMPLEMENTATION

8.1 Technology Stack

Table 8.1 lists the technologies used for each layer and the reason for each choice.

TABLE 8.1: Technology stack

Layer	Technology	Reason
Blockchain	Ethereum, Hardhat local node, Sepolia testnet [26], [6]	EVM settlement with a reproducible local target and a public testnet.
Contracts	Solidity 0.8.24, OpenZeppelin [13]	Standard token, access-control, and reentrancy primitives.
Contract tooling	Hardhat, TypeScript, Chai	Compile, deploy, test, and report gas.
Provider agent	Python 3.11, web3.py, Docker SDK [27], [28]	Chain transactions, lease and task workers, hardware discovery.
LLM runtime	Deterministic echo backend and Ollama [21]	Reproducible tests and real model serving.
Orchestrator	Node 20, TypeScript, ethers v6	Indexing, API, gateway, split and stitch.
Datastore	PostgreSQL 16 [29]	Durable projection snapshots, event cursors, replay keys.
Storage	IPFS Kubo [30]	Content-addressed manifests, inputs, outputs.
Web	Next.js 16, React 19, viem, Tailwind [31], [32]	Wallet writes through EIP-1193 [33] and indexed reads.
Compute runtime	Docker with optional NVIDIA Container Toolkit	Constrained task and lease execution with GPU access.
Dev infrastructure	Docker Compose, GitHub Actions	Local stack, provider-only node, Sepolia operator stack, CI.
End-to-end tests	Playwright [34]	Browser checks and wallet-write automation.

8.2 Provider Agent

The provider agent is a Python daemon that serves leases, serves inference, executes verifiable tasks, and answers benchmark challenges. It is untrusted by design: it signs transactions with the provider key, and the contracts decide what those transactions achieve. Table 8.2 lists its modules.

TABLE 8.2: Provider agent modules

Module	Responsibility
config.py	Environment-driven configuration.
chain.py	web3 client. Loads addresses and ABIs from the deployment file, signs and sends transactions.
storage.py	Content-addressed storage with a local directory backend for tests and an IPFS backend for the stack.
executor.py	One-shot task execution and recoverable, sandboxed lease containers with optional NVIDIA access.

Module	Responsibility
retry.py	Shared bounded exponential retry policy.
discover_hardware.py	Standard-library hardware and service discovery for Windows and Linux.
provider.py	Agent for verifiable tasks: register, stake, claim, execute, submit, challenge.
rental.py	RentalProvider (discover, accept or reject, serve, heartbeat) and RentalRenter (request, settle, end, terminate).
inference.py	OpenAI-compatible endpoint with echo and Ollama backends and token metering.
serve_provider.py	Provider node entrypoint: register, stake, list, serve inference, heartbeat, answer challenges.
benchmark.py	Deterministic challenge workload, scoring, responder, and a score command.
attester.py	Operator-run service that issues, judges, and expires challenges.

8.2.1 Lease serving

The agent accepts a pending lease only when its manifest matches an exact content-addressed manifest that the provider configured, and rejects every other request. Acceptance starts the lease and bonds stake on chain. The runtime then launches only the provider-configured image, command, environment, and device settings. It never executes arbitrary requester input. The requester cannot select the image, command, mounts, or devices. Each loop the agent checks that the workload container is healthy and sends one heartbeat for the current interval. Polling the same interval does not submit a second transaction. After a restart, the agent reconciles active leases to labelled containers.

8.2.2 Sandboxing

Lease and task containers run with no network, a read-only root file system, a non-root user, all Linux capabilities dropped, `no-new-privileges`, a small temporary file system, and limits on CPU, memory, swap, and processes. GPU access is off by default. It is enabled only when two settings agree, and it is restricted to configured devices. A task gets a size-bounded Docker volume that is mounted only at `/io`. A short-lived trusted helper writes the manifest before the untrusted task starts. No host path is mounted. The agent mounts the Docker socket to launch sibling containers, which is a host-level risk that Chapter 11 discusses.

8.2.3 Inference serving

`inference.py` serves `/v1/chat/completions` and `/v1/models`. The echo backend is deterministic and used in tests and in continuous integration. The Ollama backend

forwards to a local model server and returns the provider token counts. On start, `serve_provider.py` scans the existing offers for the provider address and keeps one active offer per configured model at the configured price. It deactivates duplicates and stale prices, and creates an offer only when needed, so a restart does not create duplicates. Offers are created only when the provider is eligible and attested. The price is configured in SMESH per 1,000 tokens and converted exactly to 18-decimal base units per token. A value that would need rounding is rejected.

8.2.4 Benchmark responder

Each poll, the agent reads its open challenge from the registry, computes the 500,000-iteration hash chain from the seed, measures a score, and calls `respondChallenge`. Two demonstration switches, `BENCHMARK_FAULT=wrong_hash` and `silent`, force the two failure paths for demonstrations.

8.3 Attester Service

`attester.py` runs with the registry attester key. For every attested provider it issues a challenge each interval (300 seconds by default), recomputes the hash chain when an answer arrives, passes the response only if the hash matches and the score reaches the required share of the attested score, and calls `expireChallenge` for missed deadlines. Its state is rebuilt from `Attested` and `ChallengeIssued` events, so a restart loses nothing. It logs one line per action in the form `challenge=<id> provider=<address> outcome=<issued|passed|failed|lapsed>`. The service never holds user funds.

8.4 Orchestrator

The orchestrator is a thin, untrusted coordinator written in TypeScript. It holds no keys and cannot move funds.

8.4.1 Event index and persistence

The indexer reads events from all six contracts and applies them in exact block and log order to an in-memory projection. Postgres stores one complete versioned snapshot and an exact (block number, log index) cursor for each deployment identity, which is the chain identifier plus the normalized contract address map. At startup the orchestrator restores the snapshot, replays only later events, starts one poll loop, and runs a second catch-up to close the replay and poll race. Every event updates and

saves a candidate snapshot and cursor before it becomes visible to API and stream consumers. The database is only a derived cache. Dropping its two tables is safe, because the next start rebuilds the projection from chain events. Historical log queries use a configurable block range. The live Sepolia indexer uses 10-block chunks to stay within the limits of the RPC provider. Transient network and rate-limit failures retry with bounded exponential backoff and degrade the health endpoint only after retries are exhausted.

8.4.2 Inference gateway

`POST /v1/chat/completions` accepts the normal OpenAI-style fields plus an authorization object. The requester signs a canonical string with EIP-191 `personal_sign` [23]. The string covers the credit identifier, expiry, the keccak256 hash of the canonical message JSON, the model, the provider, the request identifier, and the requester. The gateway verifies the signature, checks that the credit is active with nonzero escrow and matches the signer, provider, and offer model, and consumes the request identifier atomically in Postgres so that it cannot be replayed, even across restarts. It then routes only to the provider fixed by the credit. Provider endpoints must use HTTP or HTTPS, carry no credentials, match an exact hostname allowlist, and may not redirect. Request size and provider time-out are bounded. The gateway returns the provider usage and a projected credit balance. It does not settle usage.

8.4.3 Split and stitch

For deterministic jobs, `prepare` splits an image into tiles with matching sub-regions of the complex plane, builds a manifest, and pins it to storage. The returned CID is what the requester posts on chain. After every task settles, `stitchJob` fetches each output CID, composes the final image, and pins the result. Pinning is not custody of money, so it is safe for the untrusted orchestrator to do.

8.5 Web Application

The web application is built with Next.js, React, TypeScript, viem, and Tailwind. It reads indexed state from the orchestrator and sends every write through the connected wallet. It never stores secrets.

8.5.1 Pages

Table 8.3 lists the routes.

TABLE 8.3: Web application routes

Route	Purpose
/	Landing page with protocol diagrams: lease lifecycle, token flow, escrow by interval, heartbeats, challenge track.
/dashboard	Network overview: available capacity, attested providers, streaming spend.
/rent	Browse GPU listings and request a manifest-bound lease.
/leases	Own leases with heartbeat progress, streamed spend, and lease evidence. Cancel, settle, end.
/models	Model catalog and a prompt playground using prepaid credit and signed requests.
/providers	Provider console: faucet, registration, capabilities, stake, availability, listings, offers, earnings, reputation, challenge history.
/explorer	Network token-flow figure, lease inspector, Challenges tab, and a color-coded event timeline with plain descriptions.

The page list above is also the order of the navigation sidebar. The sidebar collapses to a scrolling tab row below 768 pixels.

8.5.2 Design system

The interface follows a written design system in `web/DESIGN.md`. The design is light first, with a warm paper palette and an equal dark theme selected by a toggle and remembered per browser. Amber is the single accent and appears on primary actions, active navigation, and focus rings. Neutral surfaces carry the layout. Controls use 6-pixel radii and cards 8 pixels, with hairline borders and minimal shadows. Geist is the interface typeface and Geist Mono is used for every hash, address, and amount. Status colors are muted and differ by theme. All text pairs meet WCAG AA contrast on the background, chrome, and surface colors in both themes [24]. Motion is limited to state changes of 150 to 250 milliseconds and is disabled by the reduced-motion preference. The system prohibits neon colors, purple gradients, glassmorphism, large glows, and pill-shaped buttons. Fig. 8.1 and Fig. 8.2 show the landing page in both themes.

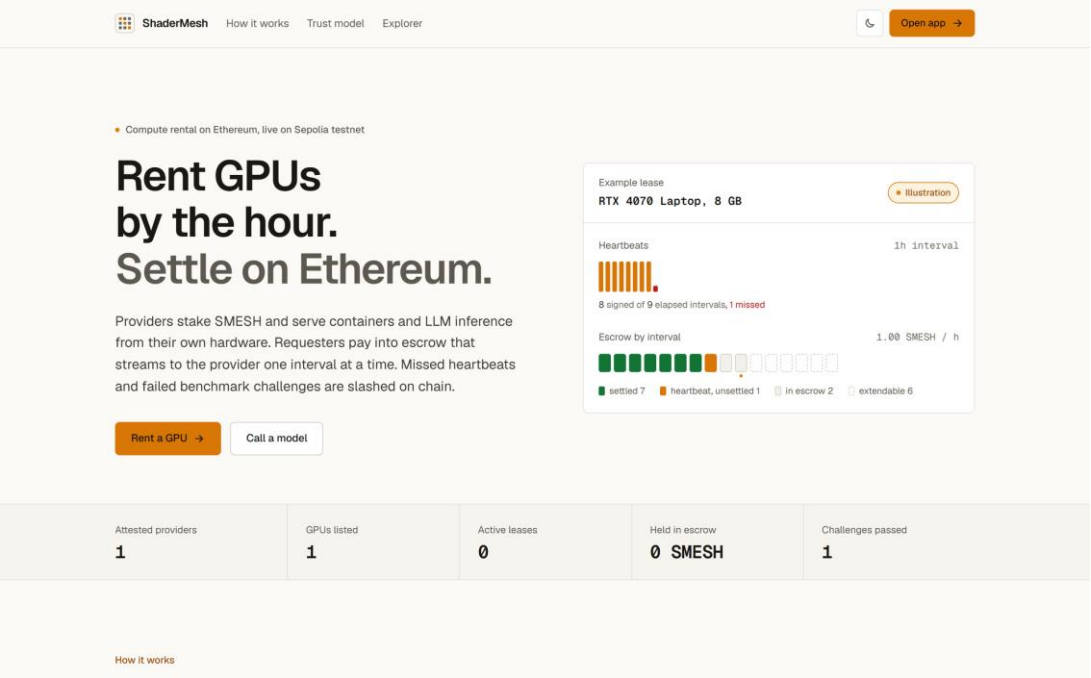


Fig. 8.1: Landing page in the light theme

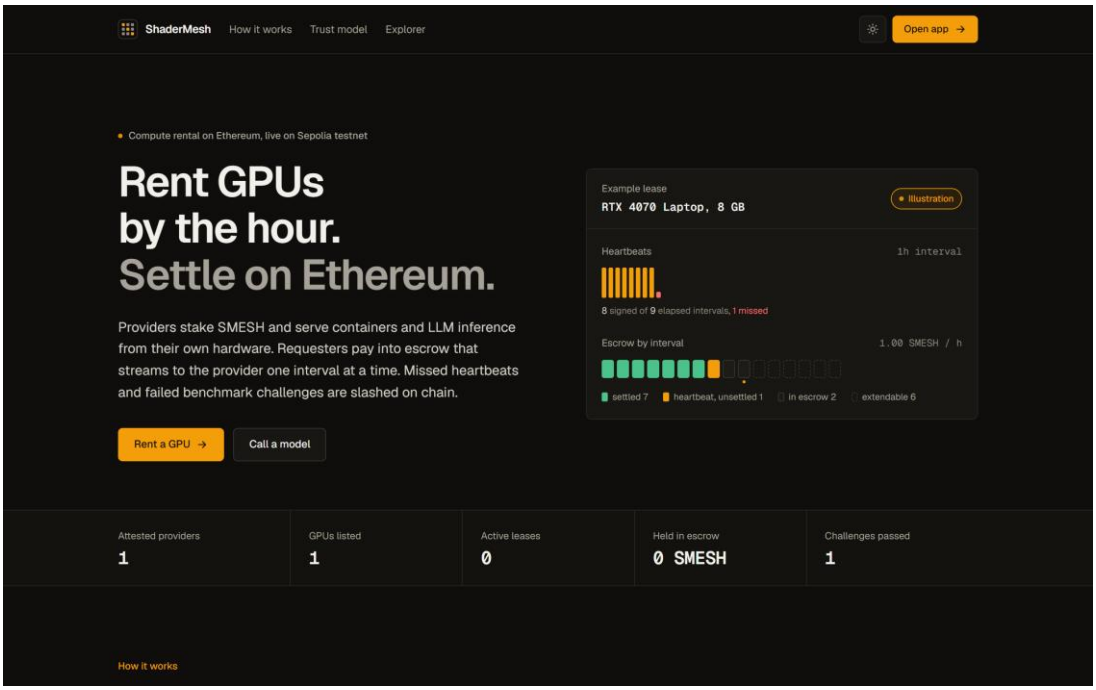


Fig. 8.2: Landing page in the dark theme

8.5.3 Wallet writes

All state changes use the connected EIP-1193 wallet. Lease requests need an ERC-20 approval followed by `requestLease`. Inference credit follows the same approve-then-open pattern. For each chat request, the page signs the canonical authorization, sends it to the gateway, and after the response submits cumulative usage to `settleUsage`. The page blocks further requests until settlement confirms or the

indexed chain state shows it completed. This prevents a requester from running ahead of its own payments.

8.6 Deployment

8.6.1 Local stack with Docker Compose

The root Compose file brings up the whole system on one machine with healthchecks and dependency conditions that enforce boot order. Its services are the Hardhat chain, the contract deployer, IPFS, Postgres, the orchestrator, a lease workload image, the provider, a seed service, the attester, and the web application. The deployer writes a descriptor with the network name, chain identifier, and each contract address and ABI to a shared volume, and off-chain services load everything from that one file. The seed service creates listings and an accepted lease so the interface has data on first load.

```
Copy-Item .env.example .env
docker compose up --build -d --wait
docker compose ps
```

8.6.2 Provider-only node

A second Compose file runs only the provider agent against an operator deployment. On Windows, a hardware discovery command first reports the operating system, CPU, memory, NVIDIA GPU and driver, Docker, and Ollama availability without printing keys. Ollama stays on the Windows host and is reached from the container through `host.docker.internal`. GPU passthrough is off unless two settings are enabled.

8.6.3 Sepolia operator and provider topology

The public deployment separates the operator from the providers. Table 8.4 lists who runs what.

TABLE 8.4: Sepolia deployment topology

Party	Runs	Holds
Operator	Contract deployment and bootstrap scripts. Operator Compose stack: Postgres, IPFS, orchestrator, web. Attester service.	Deployer key and attester key in untracked environment files. No user funds.
Provider laptop	Provider Compose stack: agent, Docker sibling workloads, Ollama on the host.	The provider wallet key. Stake is held by the registry, not by the machine.
Requester	Browser and wallet.	Own keys and funds.
Public services	Sepolia network, Etherscan for source verification, an RPC endpoint.	Public chain state.

The operator deploys with an environment-provided key and RPC endpoint, publishes the public deployment descriptor, and bootstraps providers and a test requester by an idempotent script that funds wallets to target balances and skips matching fresh attestations. Providers register themselves first. The web build carries only public-safe configuration, and the orchestrator accepts provider endpoints only from an exact hostname allowlist. All six deployed contracts were source-verified on Etherscan with a repeatable script.

CHAPTER 9: TESTING

9.1 Strategy

Testing follows the trust model. The contracts carry the value, so they get the most detailed tests, including lifecycle and conservation tests. The off-chain components are tested for the behavior that protects users, such as sandbox controls, replay protection, and ordered replay. The simulator is tested against closed-form results. The full stack is tested end to end in a browser against a fresh local deployment. Table 9.1 summarizes the suites and their counts.

TABLE 9.1: Test suite summary

Component	Tooling	Tests	Scope
Contracts	Hardhat, Chai	42	Token, registry, jobs, disputes, pending rental acceptance, strict heartbeats, escrow conservation, prepaid inference, benchmark challenges.
Provider agent	pytest, local chain, fakes	58	Task and rental lifecycles, sandbox controls, task I/O, restart recovery, offer reconciliation, hardware parsing, retries, challenger behavior, benchmark answering and judging.
Orchestrator	Vitest, Postgres	54 (plus 1 skipped without a database)	Bounded log-range replay, durable projection, ordered replay, challenge indexing, replay protection, gateway controls, API, split, stitch.
Simulator	unittest	15	Closed-form thresholds, mixed populations, rental uptime, false heartbeats, escrow conservation, determinism.
Web	Playwright	24	20 read-only checks over 10 specs on desktop and 390-pixel mobile, plus 4 desktop wallet-write specs.

9.2 Contract Tests

The 42 contract tests are split over four files. `ShaderMesh.test.ts` has 13 tests for the token faucet cooldown, registry economics and the protection of bonded stake, provider activity, attester authorization and attestation expiry, the job claim timeout slash, the claim-submit-finalize happy path, the cheating path with slash and reopen, dispute vote rules (self, duplicate, and late votes), and the disagreement quorum. `RentalMarket.test.ts` has 13 tests for pending requests bound to a manifest, acceptance-time start, bond, and reservation, rejection and cancellation refunds, one heartbeat per interval with no retroactive coverage, settlement of the contiguous heartbeated prefix, finalization, escrow conservation across payment and early-end refund, termination with prefix payment, refund, and slash, the maximum-duration

limit, lifecycle authorization, and the attestation gate on listings. `InferenceMarket.test.ts` has 9 tests for offer validity, credits with snapshotted price, cumulative settlement with replay as a no-op, rejection of decreasing counters and overdraw, authorization of settlement and close, deactivation, repeated eligibility checks, refunds, and conservation of requester funds across concurrent credits. `BenchmarkChallenge.test.ts` has 7 tests for default treasury and slash, admin and attester restrictions, a passed challenge that refreshes attestation, a failed answer that slashes free stake and revokes attestation, a missed deadline that anyone can fail, the lapse path, and the rule that bonded stake is never slashed.

9.3 Provider Agent Tests

The agent suite of 58 tests runs against a real local Hardhat chain through shared session fixtures, with fakes for Docker and IPFS where a unit test needs them. Integration tests drive the verifiable-task lifecycle including slashing, the rental lifecycle including acceptance, rejection, streamed settlement, refund, and downtime slashing, and the challenge lifecycle against the real registry, including restart safety. Unit tests cover the lease executor (sandbox flags, GPU gating, recovery labels, cleanup, health-backed heartbeats), worker safety (task sandboxing, retry classification and backoff, IPFS retries, challenger sampling), inference (metering, exact offer pricing, idempotent offer creation), benchmark determinism and judging, and hardware parsing.

9.4 Orchestrator Tests

The orchestrator suite has 54 tests that run in continuous integration against Postgres, and one Postgres integration test that is skipped when `TEST_DATABASE_URL` is not set. Tests cover API routes, the authorization canonicalization and signature checks, the gateway controls (body limit, time-out, hostname allowlist, redirect refusal, replay), challenge indexing, the indexer (ordered replay and bounded range), persistence of the projection, rental and inference projections, splitting, stitching, and the store.

9.5 Simulator Tests

The 15 simulator tests check the closed-form break-even values, the rental false-heartbeat economics, escrow conservation, determinism by seed, mixed population accounting, full uptime, immediate downtime detection, undetected false heartbeats, behavior with no challengers, and rejection of invalid inputs.

9.6 End-to-End Browser Tests

Playwright runs 24 checks against the running Compose stack. It does not start or stop services. There are two projects: a desktop Chromium viewport of 1440 by 900 pixels and a mobile viewport 390 pixels wide. Ten read-only specs run on both and check that each route renders cleanly with no browser errors or horizontal overflow, that navigation reaches every route, that the explorer tabs expose indexed sections, that the landing page opens the app, and that the theme toggle switches to dark and persists. This gives 20 read-only checks.

9.6.1 Wallet-write automation

Four further specs perform real wallet writes and run on desktop only, because they share one chain and one listing. A real browser wallet cannot be driven reliably in continuous integration, so the specs inject a minimal EIP-1193 provider [33] into the page. The provider forwards JSON-RPC calls to the local Hardhat node, which signs for its unlocked accounts. This exercises the real application code, real contract calls, and the real orchestrator, and replaces only the wallet user interface. The four specs are:

1. Connect a wallet and show the address and the SMESH balance.
2. Request a lease, observe it become Active, show heartbeat evidence, end it, and check the refund.
3. Open a prepaid inference credit, send a chat request with a signed authorization, settle usage, and close the credit.
4. Register and stake a fresh provider and show it as unattested.

The specs use Hardhat accounts 5, 6, and 7 and a new random address for the provider spec. If every listing is reserved by another lease, the fixture ends or cancels that lease first. The specs must not be run against a public network.

9.7 Continuous Integration

GitHub Actions runs five component jobs and a dependent full-stack job on every push and pull request. The contracts job compiles, tests, and records gas. The agent job runs the Python suite. The orchestrator job runs all tests against a Postgres service and type checks. The web job runs lint, type check, and a production build. The simulator job runs its tests and the simulation. The full-stack job starts a fresh

Compose deployment with the echo inference backend, runs the signed prepaid-inference smoke flow, and runs the 24 Playwright checks.

9.8 Gaps

Property-based fuzzing of the contracts is not present. Escrow conservation is covered by lifecycle tests for the current paths, but not by randomized sequences. A formal accessibility audit, a measurement of event latency, and cross-platform runs beyond Windows development and Linux continuous integration are also not complete. The contracts have not been externally audited.

CHAPTER 10: RESULTS AND EVALUATION

10.1 Evaluation Setup

The evaluation has three parts. The gas report measures the cost of each core transaction on the Hardhat test chain. The simulator evaluates the economic mechanisms with a deterministic Monte Carlo model using seed 42. The live deployment on Sepolia shows the system working with real hardware. All numbers in this chapter come from the recorded outputs `docs/_report/data/gas.txt` and `docs/_report/data/simulator.txt`, or from the live deployment records, and can be reproduced with the commands in Appendix C.

10.2 Gas Costs

Table 10.1 lists the gas used by each core transaction, taken from the transaction receipts of the test suite. The measurement covers 27 transactions with a combined cost of 3,086,966 gas. Fig. 10.1 charts the same data with the benchmark challenge calls highlighted.

TABLE 10.1: Gas used by core transactions

Contract	Function	Gas used
SmeshToken	faucet	76,533
ProviderRegistry	register	91,325
ProviderRegistry	stake	81,667
ProviderRegistry	updateCapabilities	32,175
ProviderRegistry	attest	97,252
JobMarket	postJob	179,415
JobMarket	claimTask	159,503
JobMarket	submitResult	83,491
JobMarket	finalizeTask	102,576
JobMarket	claimTask (disputed)	159,515
JobMarket	submitResult (disputed)	83,503
Verification	challenge	179,522
Verification	resolve	129,700
RentalMarket	createListing	139,228
RentalMarket	requestLease	251,540
RentalMarket	acceptLease	154,207
RentalMarket	heartbeat	56,487
RentalMarket	settle	59,987
RentalMarket	finalizeExpired	88,634
ProviderRegistry	issueChallenge	129,754

Contract	Function	Gas used
ProviderRegistry	respondChallenge	76,217
ProviderRegistry	resolveChallenge (pass)	54,309
ProviderRegistry	expireChallenge (slash)	80,193
InferenceMarket	createOffer	134,737
InferenceMarket	openCredit	227,898
InferenceMarket	settleUsage	124,492
InferenceMarket	closeCredit	53,106
Total	27 transactions	**3,086,966**

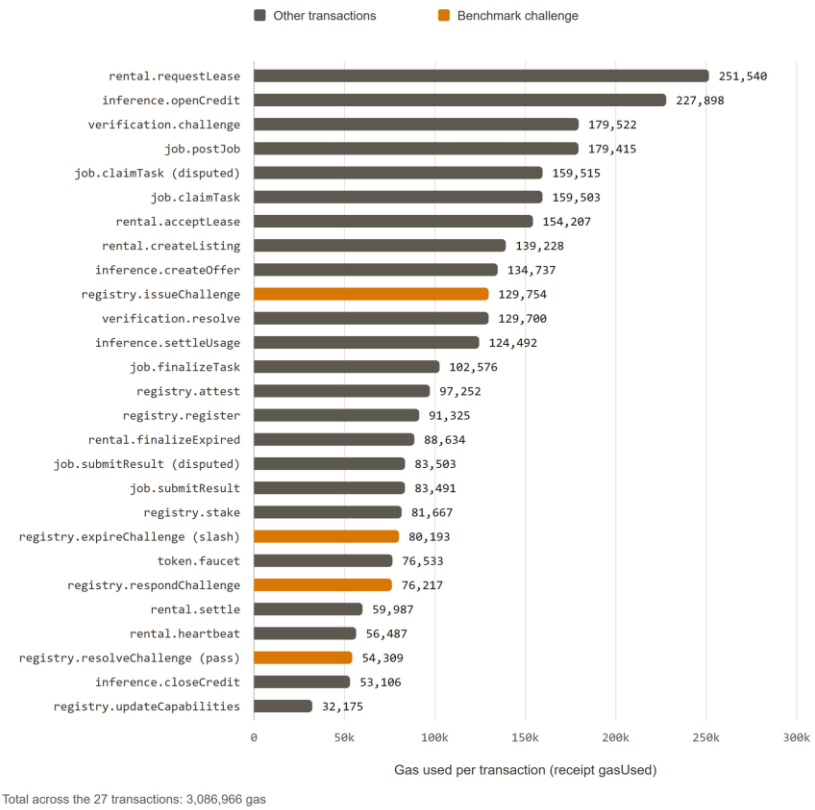


Fig. 10.1: Gas per core transaction, with benchmark challenge calls highlighted

Four observations follow from the data.

1. **Opening a position costs more than operating it.** The most expensive transactions are `requestLease` (251,540 gas), `openCredit` (227,898), `verification.challenge` (179,522), and `postJob` (179,415). Each creates a record that holds a manifest or escrow. Recurring calls are cheaper: `heartbeat` costs 56,487, `settleUsage` 124,492, and `closeCredit` 53,106.
2. **Whole-flow costs are modest.** A one-interval lease that is requested, accepted, heartbeated, settled, and finalized costs 610,855 gas across the requester and the provider. A prepaid inference session with one settlement and a close costs

405,496 gas after the offer exists. A single-task job from post to finalization costs 524,985 gas.

3. **The heartbeat is the recurring provider cost.** At 56,487 gas per interval, a provider that heartbeats every 60 seconds, as in the live deployment, pays this cost 60 times an hour. That is acceptable on a test network and would be significant on mainnet. Relayed or batched signed heartbeats are planned for this reason.
4. **A benchmark challenge is cheap for what it checks.** A passed challenge (issue, respond, resolve) costs 260,280 gas in total, and the permissionless expiry that slashes a silent provider costs 80,193. The attester pays for issue and resolve. The provider pays for the response (76,217).

Gas is a quantity of work. The currency cost depends on the gas price. As an illustration only, at an assumed price of 1 gwei, `requestLease` would cost 0.00025154 ETH. No fee measurement on Sepolia is claimed here.

10.3 Verifiable-Task Simulation

The simulator models the optimistic verification game in two settings. The first checks the closed-form break-even rate of Chapter 6 with a single type of cheater. The second uses a mixed population of providers. The parameters were reward 100, bond 100, three challengers, and 100,000 rounds.

10.3.1 Break-even spot-check rate

Table 10.2 compares the simulated expected value of cheating with the closed form for eight spot-check rates. The break-even rate is $s^* = R / (R + B) = 0.500$. The simulated values track the closed form within 0.4 units in every row. At $s = 0.5$ the simulated value is -0.05 against a theoretical 0, so cheating stops being profitable at the predicted point. Fig. 10.2 plots the result.

TABLE 10.2: Expected value of cheating by spot-check rate (reward 100, bond 100)

s	Detection	Overhead	EV simulated	EV theory	Verdict
0.000	0.0%	0.000	100.00	100.00	profitable
0.100	10.0%	0.301	79.96	80.00	profitable
0.170	17.1%	0.514	65.75	66.00	profitable
0.200	20.2%	0.605	59.64	60.00	profitable
0.300	30.1%	0.902	39.84	40.00	profitable
0.500	50.0%	1.501	-0.05	0.00	deterred
0.750	74.9%	2.248	-49.83	-50.00	deterred
1.000	100.0%	3.000	-100.00	-100.00	deterred

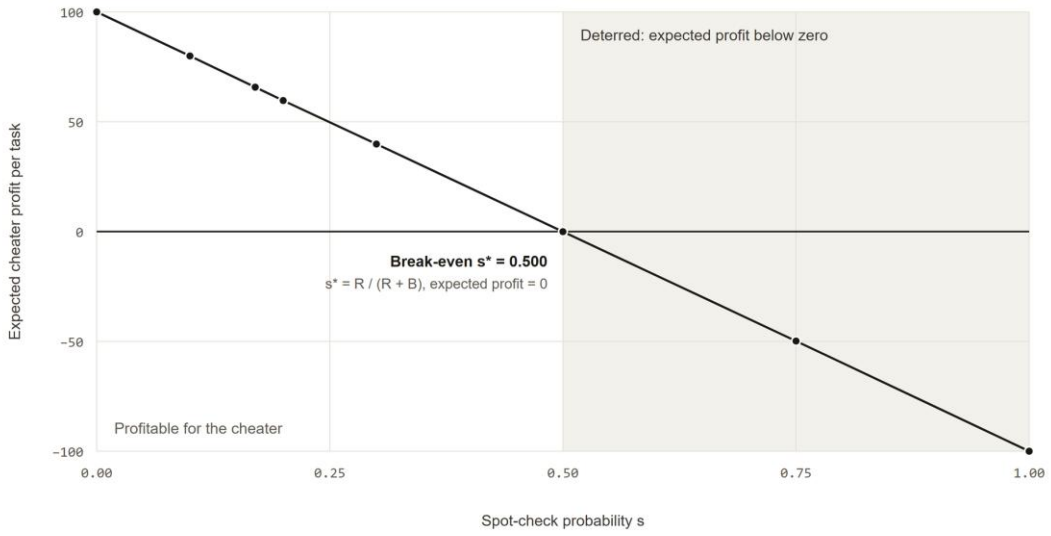


Fig. 10.2: Expected cheater profit against spot-check probability (verifiable task mode)

The overhead column is the number of extra executions per task, which equals the spot-check rate times the number of challengers. At the break-even rate, deterrence costs 1.5 extra executions per task with three challengers. A larger bond lowers s* and therefore the overhead, which is the design lever that the analysis predicts.

10.3.2 Mixed provider population

The second experiment uses 60 percent honest, 20 percent lazy, 10 percent malicious, and 10 percent colluding providers, a spot-check rate of 0.5, and three challengers. Table 10.3 shows the outcome by provider type.

TABLE 10.3: Task verification outcomes in a mixed population

Type	Tasks	Cheats	Caught	Detection	Cheat profit or loss
honest	59,754	0	0	0.0%	0.00
lazy	19,971	9,984	3,847	38.5%	229,000.00
malicious	10,036	10,036	3,875	38.6%	228,600.00
colluding	10,239	10,239	3,295	32.2%	364,900.00

Overall, 30,259 cheats occurred and 36.4 percent were detected, at an overhead of 1.501 extra executions per task. This result differs from the closed-form case in an important way. With a spot-check rate of 0.5, a perfect check would detect half of all cheats. The observed rates are 32 to 39 percent, because detection needs a strict majority of correct challenger votes. With three challengers drawn from a population in which only 60 percent are honest, a majority is correct less often than not. The last column is positive for every cheating type, which means cheating remained profitable in aggregate for this mix and these parameters.

The result supports two items in the planned hardening. The break-even formula assumes that a spot check detects a cheat whenever it happens. When challengers are drawn from a mixed pool, the effective detection rate is lower, so the system needs random and reputation-weighted challenger selection, more challengers, or a higher bond. The current implementation volunteers challengers and applies a quorum of one, so this simulation should be read as a warning about the unhardened design and not as a measurement of it.

10.4 Rental Simulation

The rental simulation runs 10,000 leases of 24 intervals each, with an interval price of 10, a bond of 500, and a probability of 0.75 that a false heartbeat is detected. Honest providers are up 99 percent of intervals, lazy providers 80 percent, malicious providers 50 percent, and colluding providers 70 percent. A provider that is down either omits the heartbeat, which is direct evidence, or sends a false heartbeat, which is caught with the detection probability. The first detected failure terminates the lease, refunds unpaid intervals, and slashes the bond. The false-heartbeat break-even detection rate is $h^* = V / (V + D) = 0.020$. Table 10.4 and Fig. 10.3 give the results.

TABLE 10.4: Simulated rental outcomes by provider type

Type	Leases	Uptime	Detect	Paid	Refunded	Slashed	Terminated
honest	6,052	99.0%	100.0%	1,286,790.00	165,690.00	656,000.00	21.7%
lazy	1,980	79.5%	100.0%	76,410.00	398,790.00	986,000.00	99.6%
malicious	999	52.1%	75.7%	17,540.00	222,220.00	499,500.00	100.0%
colluding	969	69.6%	73.4%	33,620.00	198,940.00	482,000.00	99.5%

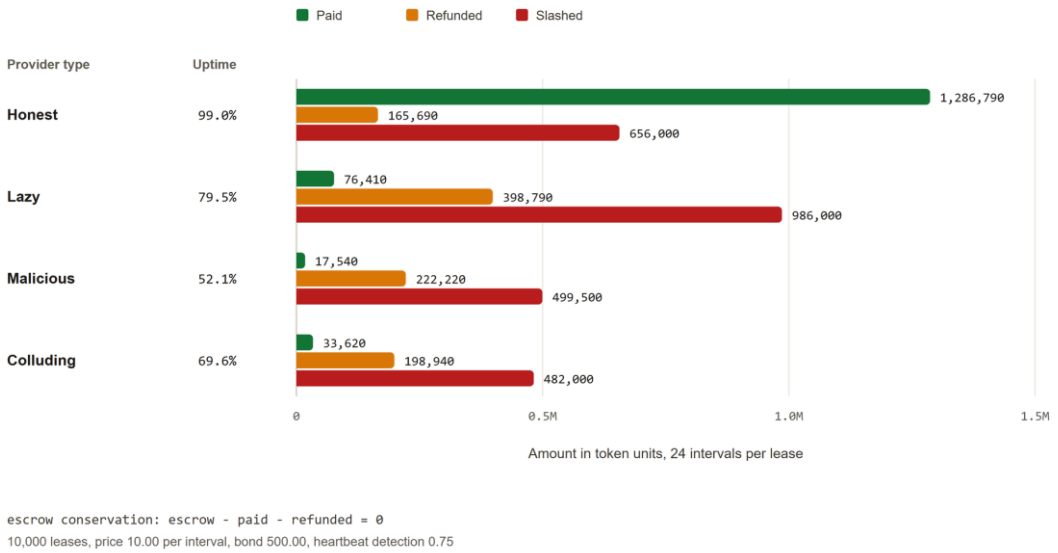


Fig. 10.3: Simulated rental outcomes by provider type: paid, refunded, and slashed

Escrow conservation holds. Funded escrow was 2,400,000, provider payment was 1,414,360, and requester refunds were 985,640. Payment plus refund equals escrow exactly, so no funds were created or lost. Slashed collateral of 2,623,500 is accounted separately, because it comes from provider stake and not from requester escrow.

Three results stand out.

- **Providers that fail are paid little and lose their bond.** Lazy, malicious, and colluding providers received between 17,540 and 76,410 in payment across their leases, and almost every one of their leases ended in termination (99.5 to 100 percent).
- **Detection is complete for omitted heartbeats and partial for false ones.** Honest and lazy providers show 100 percent detection because their down intervals are direct evidence. Malicious and colluding providers send false heartbeats and show 75.7 and 73.4 percent, consistent with the 0.75 setting.
- **Strict termination also hits honest providers.** Even with 99 percent uptime, 21.7 percent of honest leases ended in termination and 656,000 in bond was

slashed from honest providers. With 24 intervals, the chance of at least one missed interval is 1 - 0.99^24, about 21 percent. The model terminates a lease at the first miss, with no tolerance. The contract adds a grace period of five minutes after the interval end, but it also terminates on the first miss that passes the grace period. The result is a design signal: the grace period and any tolerance for brief outages are first-order parameters, and a production design would likely need a small outage budget or a proportional penalty.

10.5 Live Sepolia Results

10.5.1 Deployment

The system was redeployed to Sepolia (chain identifier 11155111) on 2026-10-24 with the benchmark challenge feature and the attester service. All six contracts are source-verified on Etherscan with a repeatable script. Their addresses are in Appendix A.

10.5.2 Provider and challenge

The live provider `0xfd4e4A034601991A3070bD2B9B0F6a473a5bfc76` runs on a laptop with an NVIDIA GeForce RTX 4070 Laptop GPU with 8 GB of memory. It registered, staked 100 SMESH, was attested, created rental listing #1 with 60-second intervals, and created inference offer #1 for `qwen3:0.6b`. It then passed live benchmark challenge #1. The measured score was 28,919, which is about 289 tokens per second on `qwen3:0.6b` (the score is tokens per second times 100). The provider answered 108 seconds after issuance, well inside the 15-minute challenge window used on Sepolia. A pass raised the provider reputation from 5,000 to 6,000. The provider console and the explorer show this record in Fig. 10.4 and Fig. 10.5.

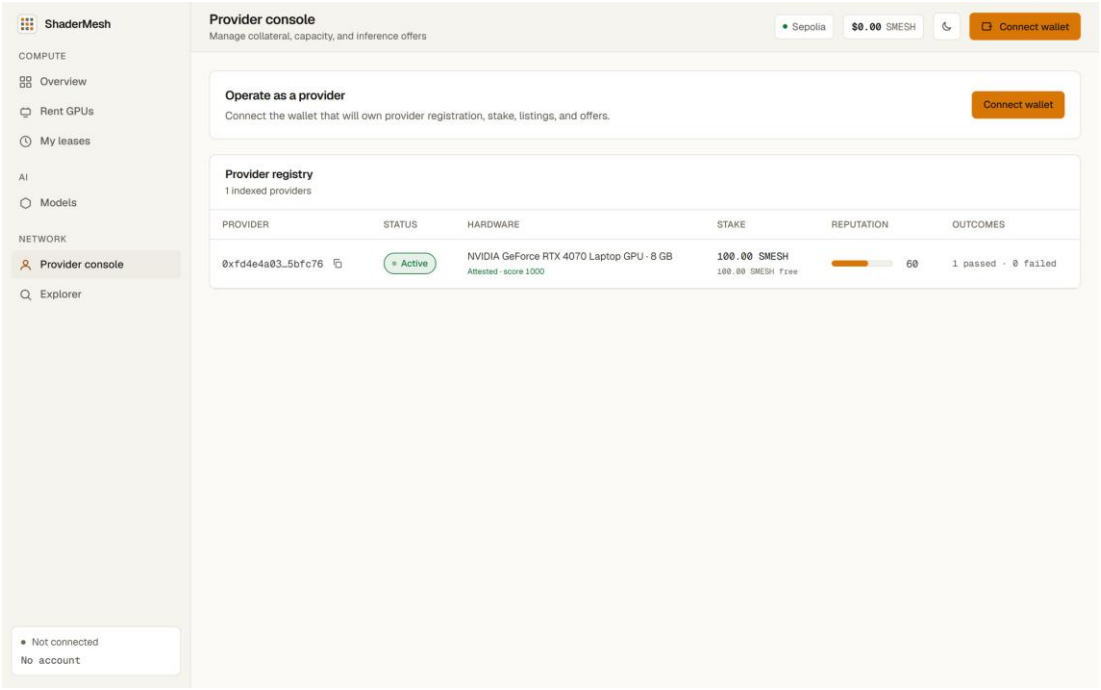


Fig. 10.4: Provider console on Sepolia: the RTX 4070 Laptop GPU provider with 100 SMESH of stake, attested, one challenge passed

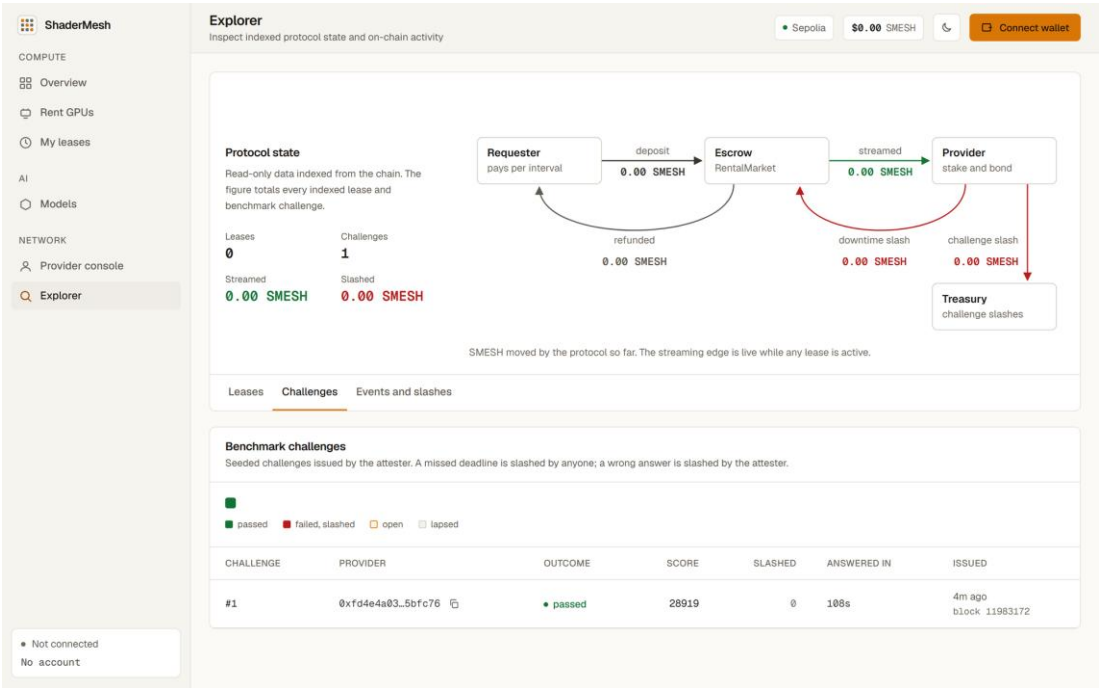


Fig. 10.5: Explorer Challenges tab on Sepolia: benchmark challenge #1 passed with score 28919, answered in 108 s

The attested benchmark score of this provider is 1,000, so the 28,919 measurement far exceeds the required 50 percent. The attested score was set low on purpose so that both the echo backend and a real model pass reliably. A tighter attested score is a deployment choice and does not change the mechanism.

10.5.3 Flows validated on the Review 1 deployment

Before the redeployment, the earlier Sepolia deployment validated the main user flows on the same provider. These records are kept in a deployment file that is not committed, and they are reported here as the earlier validation, not as a result of the current addresses.

- **Prepaid inference.** A real `qwen3:0.6b` request served 36 tokens through a prepaid credit and settled 72,000,000,000,000 SMESH base units to the provider. The price was 0.002 SMESH per 1,000 tokens, so 36 tokens cost 0.000072 SMESH.
- **GPU lease.** A manifest-bound lease launched an immutable CUDA container on the GPU, recorded two heartbeats, settled one interval, and ended normally.
- **Deterministic jobs.** Two jobs executed in isolated sibling containers, submitted matching commitments, settled after the challenge window, and were stitched to a single IPFS result.
- **Operations.** All operator and provider containers reported healthy, and the indexer ran with 10-block log queries on the RPC provider tier used.

10.5.4 Application screens

The following figures show the web application running against the live Sepolia deployment. Fig. 10.6 shows the overview, Fig. 10.7 the rental page, Fig. 10.8 the model catalog, Fig. 10.9 the explorer lease inspector, Fig. 10.10 the event timeline, and Fig. 10.11 the Challenges tab in the dark theme. The screenshots were taken without a connected wallet.

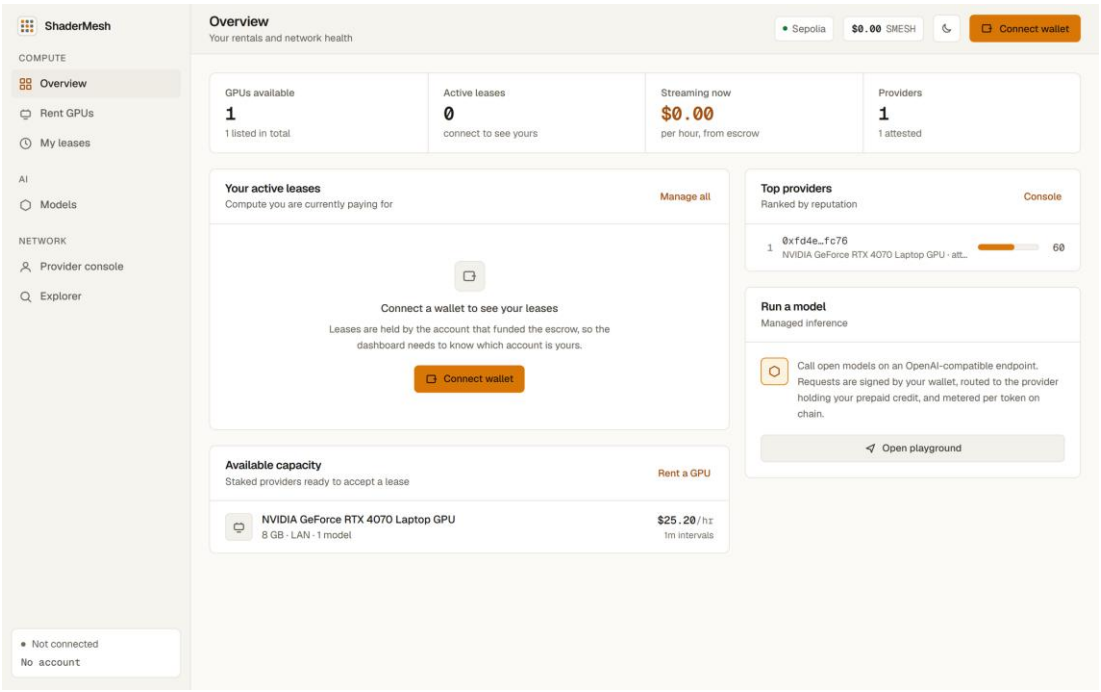


Fig. 10.6: Overview page with live network figures

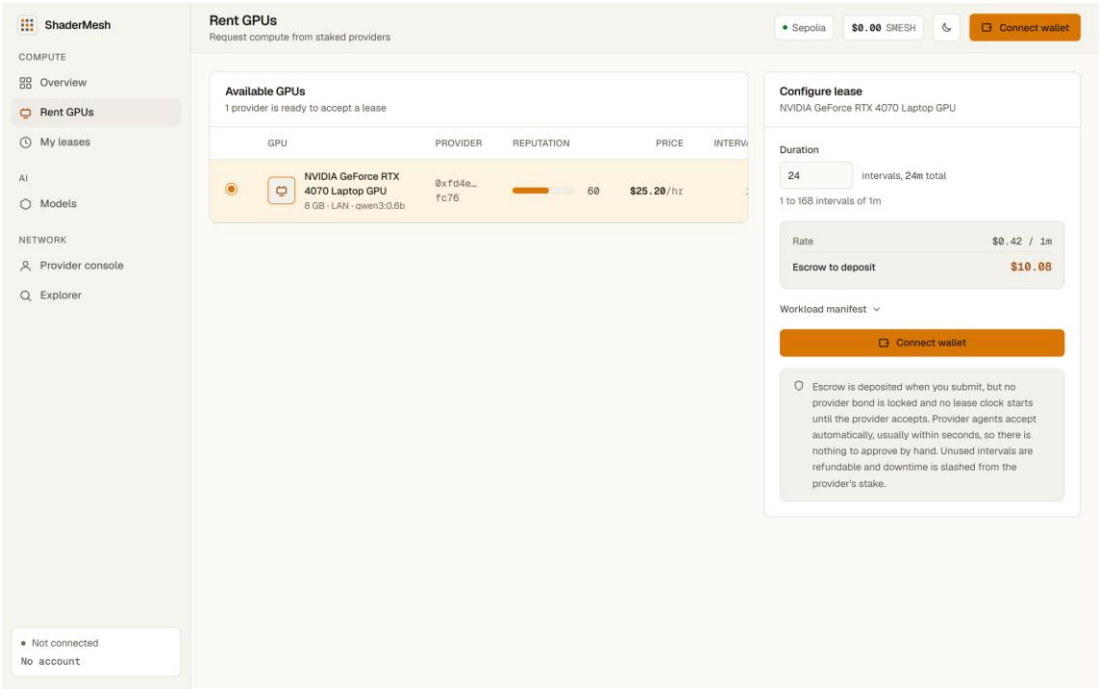


Fig. 10.7: Rent GPUs page listing available capacity

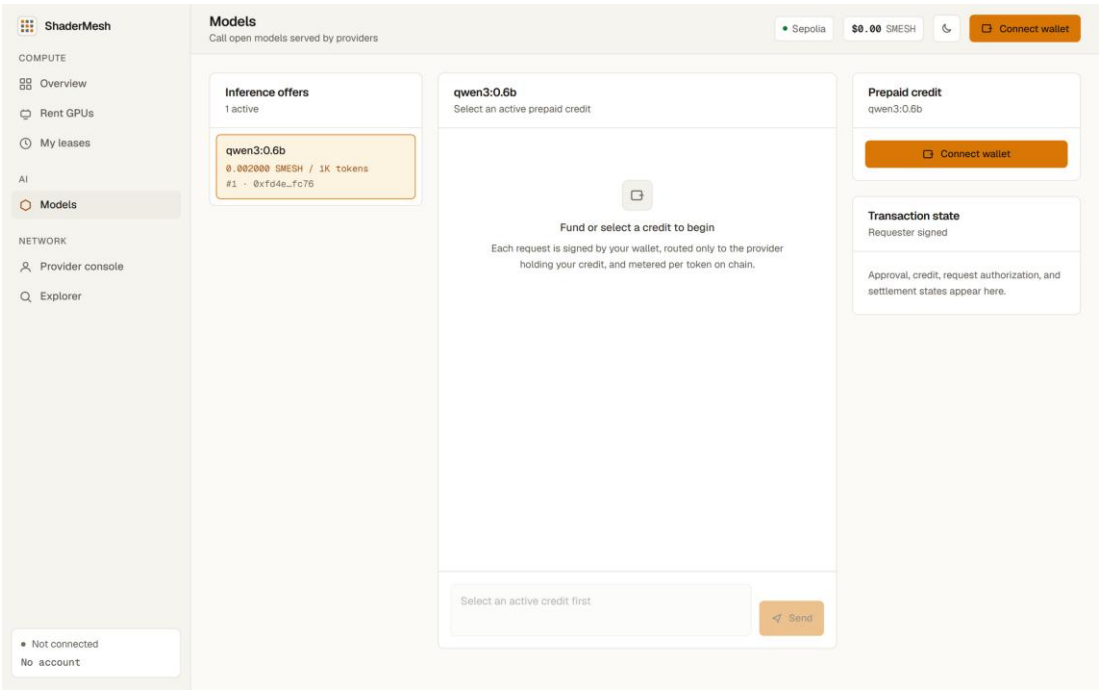


Fig. 10.8: Models page with the inference offers from the live provider

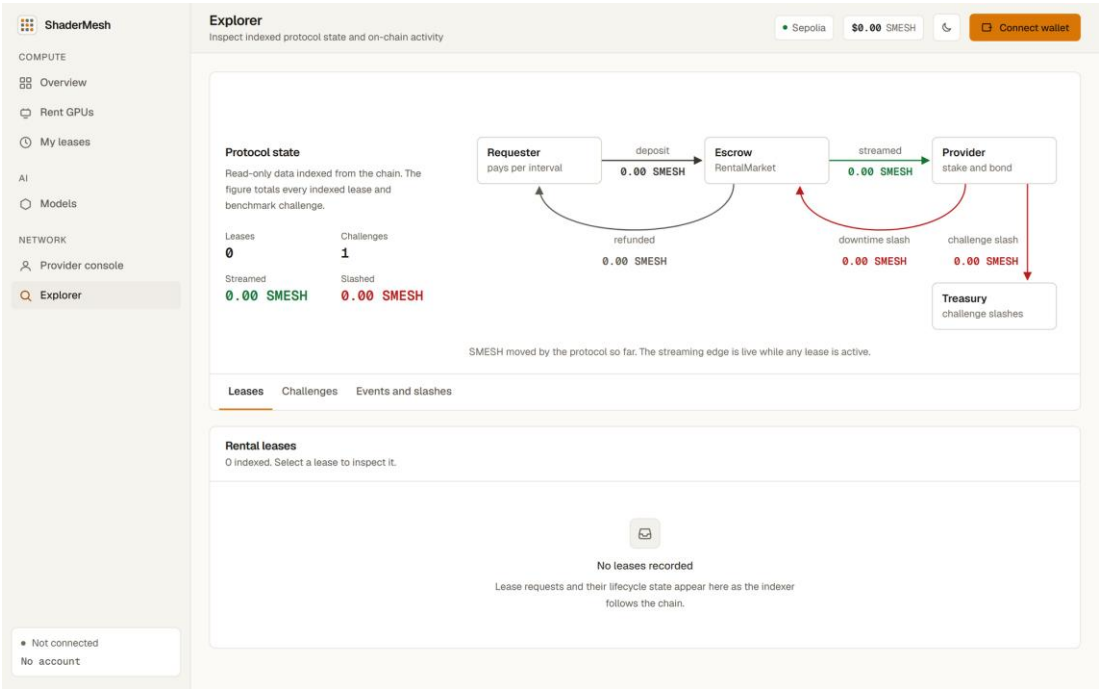


Fig. 10.9: Explorer Leases tab with the lease inspector

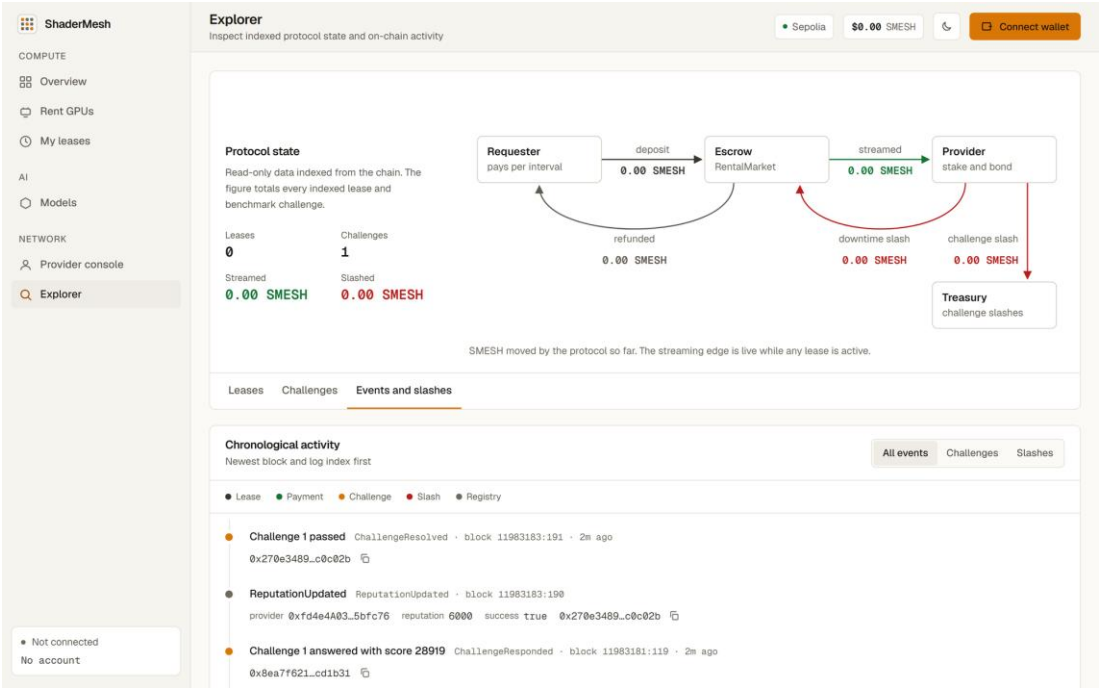


Fig. 10.10: Explorer events and slashes timeline

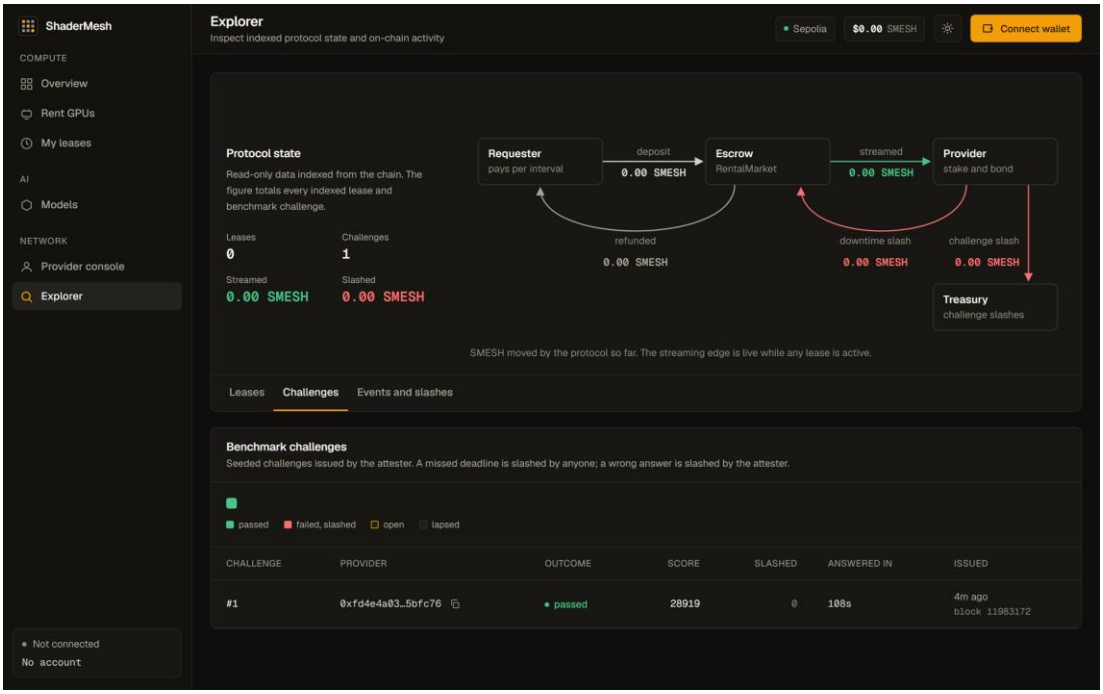


Fig. 10.11: Explorer Challenges tab in the dark theme

10.6 Summary Against Objectives

The results support the five objectives of Chapter 2. The contracts implement streamed escrow, bonded stake, and slashing, with 42 passing tests and modest gas costs. The rental model is secured by heartbeats and benchmark challenges instead of result checking, and the simulator shows that failing providers are paid little and lose their bond while escrow is conserved. LLM and GPU compute were served on a real

provider. The verifiable-task market works with exact hashes, and the simulation shows that its deterrence depends on the quality of the challenger pool. The prototype is live on a public testnet with one physical provider.

CHAPTER 11: LIMITATIONS, RISKS, AND ETHICAL CONSIDERATIONS

11.1 Limitations

The following limits apply to the delivered system. Several are inherent to the problem, and the report states them so the results are not over-read.

11.1.1 The attester is a trusted judge

The attester writes attestations and judges benchmark answers. The registry bounds what it can do: it can slash only free stake, only up to a fixed amount per challenge, and only to the treasury. It cannot be used to drain lease bonds. It can still fail a correct answer, and it controls re-attestation. The system therefore relies on the attester being operated by a party that the participants accept. Decentralizing the attester, for example with several independent attesters and a threshold, is future work.

11.1.2 LLM output is not verified on chain

The chain cannot judge whether a model answer is correct. The inference market checks usage counters and payment, not the content of responses. A provider could serve a smaller model than advertised, and the current system would detect this only through requester action. Sampled, deterministic spot checks that feed reputation are planned, and they could not be used for hard slashing because outputs differ across hardware.

11.1.3 A heartbeat proves liveness of a key, not quality of service

A heartbeat is a transaction signed by the provider key. It shows that the key was live and that the agent considered the workload healthy. It does not show that the workload was fast, that the GPU was the advertised model, or that the requester could reach the service. Independent probes that measure availability from outside the provider are planned.

11.1.4 A benchmark does not prove the GPU model

A benchmark proves responsiveness, key control, and throughput near an attested value. A different, adequate GPU would pass. The system does not claim stronger hardware identity than that.

11.1.5 Requester-metered inference

The requester submits the cumulative token counts. The contract prevents overdraw and decreasing counters, but it cannot prevent under-reporting, which would underpay the provider. The requester has an economic reason to under-report. The current design is suitable for a cooperative setting. Provider-signed usage receipts, or a bounded dispute process, are needed before production use.

11.1.6 Single live provider and testnet only

The public deployment has one physical provider. The results show that the mechanisms work with real hardware, but they do not show market behavior with many competing providers. The system runs only on a test network. SMESH has no value, and the economic parameters were not tuned against real incentives.

11.1.7 Task verification is not fully hardened

Challengers volunteer and are not selected at random. Votes are plaintext. The quorum is one. The simulation in Chapter 10 shows that detection falls below the nominal spot-check rate when the challenger pool is mixed. Commit-reveal, reputation-weighted random selection, and perceptual comparison for rendering remain planned.

11.1.8 Other limits

- Lease workloads are provider-configured. A requester cannot run an arbitrary container, which limits flexibility and is a deliberate safety choice.
- The gateway routes to the provider fixed by the credit. It does not probe health or fail over.
- Reputation is an exponential moving average of outcomes. It is not proportional to uptime and has no lease dispute process.
- The strict heartbeat termination rule penalizes brief outages heavily, as the rental simulation shows.
- The contracts have not been externally audited and are not fuzz tested.

11.2 Risks

Table 11.1 lists the main risks, their impact, and the mitigation in the current system.

TABLE 11.1: Risk register

Risk	Impact	Mitigation and residual risk
Docker socket access on the provider agent	Host compromise from a malicious workload	Workloads are provider-configured. Containers have no network, read-only root, dropped capabilities, and resource limits. The socket is a host-level authority. Run the agent on a dedicated host.
Provider spoofs advertised hardware	Requester pays for less than promised	Attestation gate and recurring seeded benchmarks. Residual risk: trusted judge, and a benchmark does not prove the GPU model.
Collusion among providers in task verification	Incorrect result accepted	Majority voting assumes honest participants. Selection and weighting are planned.
Requester under-reports inference usage	Provider underpaid	Explicit in the design. Receipts or disputes are future work.
Testnet and RPC variability	Unreliable public demonstration	Local devnet fallback. Small log-query chunks on the RPC tier used.
Escrow accounting errors	Funds locked or mispaid	Lifecycle and conservation tests. Property-based fuzzing remains a gap.
GPU and runtime setup on varied hosts	Operational friction	Hardware discovery command and a CPU-served small model fallback.
Operator key compromise (attester or owner)	Unfair slashing or parameter changes	Test network only. Production needs multi-signature control and key separation.

11.3 Ethical Considerations

- **No financial instrument.** SMESH is a utility and test token. The project does not sell tokens, offer returns, or custody user funds. The README and the interface do not present SMESH as an investment.
- **Confidentiality of renter data.** Ordinary Docker isolation is not suitable for highly confidential data. A provider can observe what runs on its machine. The system does not claim confidentiality for requester data, and renters should not place sensitive data on a public provider.
- **Public ledger.** Addresses, manifests, and payment records on a public chain are visible to everyone and cannot be erased. Users should treat wallet addresses as pseudonymous but linkable.
- **Misuse of compute.** Renting capacity to strangers could allow harmful workloads. The current design reduces this risk by running only provider-approved images, with no network. A system that accepts arbitrary requester containers would need stronger isolation and abuse handling.

- **Provider safety.** Providers bear security risk on their own machines. The documentation tells providers to use dedicated wallets, test funds, and a dedicated host.
- **Secrets and responsible testing.** No private keys or RPC credentials are committed. Only test wallets and test ETH were used. Wallet-write tests are never run against a public network.
- **Concentration of control.** The owner and attester roles are privileged. Decentralized marketplaces can still concentrate power in their operators, and this limit is stated rather than concealed.
- **Energy and resources.** Sepolia uses proof of stake and the compute load of the project is small. The environmental impact is limited to the laptop and the test servers used.

CHAPTER 12: CONCLUSION AND FUTURE WORK

12.1 Conclusion

ShaderMesh shows that a compute-rental marketplace can keep value and enforcement on Ethereum while leaving compute off chain. The key design decision was to use two explicit trust models. Rentals, which cannot be re-executed, are secured by streamed escrow, bonded stake, strict per-interval heartbeats, attestation, and seeded benchmark challenges. Deterministic jobs, which can be re-executed, use an optimistic verification game. Both share one registry for stake, reputation, and slashing.

The delivered system includes six contracts with 42 tests, a provider agent with 58 tests, an orchestrator with 54 tests, a simulator with 15 tests, and a web application with 24 browser checks that include wallet writes. The system runs locally with one Compose command and is deployed and source-verified on Sepolia. A real provider with an RTX 4070 Laptop GPU registered, staked, was attested, listed capacity, served a real model, and passed a live benchmark challenge. The gas report shows modest costs for the core flows. The simulator reproduces the break-even spot-check rate of 0.5 for equal reward and bond and confirms escrow conservation in rentals.

The evaluation also exposed design signals. Detection in a mixed challenger population falls below the nominal spot-check rate. Strict heartbeat termination penalizes honest providers with brief outages. The heartbeat transaction dominates the recurring provider cost. These findings guide the future work below and are part of the contribution, because they come from running the mechanism rather than describing it.

12.2 Future Work

12.2.1 Protocol

- Commit-reveal voting, random and reputation-weighted challenger selection, and a higher quorum for the task market.
- Relayed or batched signed heartbeats to reduce provider gas, and independent availability probes.
- A tolerance or outage budget for heartbeats, and a reputation score proportional to uptime with a lease dispute process.

- Multiple independent attesters with a threshold, to remove the single trusted judge.
- Provider-signed usage receipts or a bounded dispute process for inference metering.
- Typed CPU and region fields and a structured context length in offers.

12.2.2 Verification of workloads

- Sampled deterministic inference spot checks that feed reputation.
- Blender rendering with perceptual (SSIM) comparison and distributed tile assembly. This was descoped in the final phase plan.

12.2.3 Platform

- Gateway health probing and failover across providers.
- Requester-defined containers with stronger isolation, such as user-space kernels or micro virtual machines.
- Verifiable-job submission and live tile assembly in the web application.
- More provider hosts and a longer public run to study market behavior.

12.2.4 Assurance

- Property-based fuzzing of escrow and bond accounting, and an external audit before any use beyond a test network.
- Formal accessibility testing and measurement of event latency in the dashboard.
- Deployment on an EVM layer 2 network to lower costs.

REFERENCES

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. Accessed: Oct. 2026. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] V. Buterin, "A next-generation smart contract and decentralized application platform," Ethereum white paper, 2014. Accessed: Oct. 2026. [Online]. Available: https://ethereum.org/en/whitepaper/

[3] OpenAI, "API reference: Chat completions." Accessed: Oct. 2026. [Online]. Available: https://platform.openai.com/docs/api-reference/chat

[4] F. Vogelsteller and V. Buterin, "EIP-20: Token standard," Ethereum Improvement Proposals, 2015. Accessed: Oct. 2026. [Online]. Available: https://eips.ethereum.org/EIPS/eip-20

[5] Ethereum Foundation, "Proof-of-stake (PoS)," ethereum.org developer documentation. Accessed: Oct. 2026. [Online]. Available: https://ethereum.org/en/developers/docs/consensus-mechanisms/pos/

[6] Ethereum Foundation, "Networks: Sepolia testnet," ethereum.org developer documentation. Accessed: Oct. 2026. [Online]. Available: https://ethereum.org/en/developers/docs/networks/

[7] ShaderMesh project team, "ShaderMesh: Review 1 proposal and report," project repository, `docs/ShaderMesh-Review1-Proposal.md`, Sep. 2026, unpublished.

[8] Vast.ai, "Vast.ai documentation." Accessed: Oct. 2026. [Online]. Available: https://docs.vast.ai/

[9] J. Teutsch and C. Reitwiessner, "A scalable verification solution for blockchains," arXiv:1908.04756, 2019. Accessed: Oct. 2026. [Online]. Available: https://arxiv.org/abs/1908.04756

[10] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten, "Arbitrum: Scalable, private smart contracts," in Proc. 27th USENIX Security Symp., Baltimore, MD, USA, 2018. Accessed: Oct. 2026. [Online]. Available: https://www.usenix.org/conference/usenixsecurity18/presentation/kalodner

[11] D. Petkanics and E. Tang, "Livepeer whitepaper: Protocol and economic incentives for a decentralized live video streaming network," 2017. Accessed: Oct. 2026. [Online]. Available: https://github.com/livepeer/wiki/blob/master/WHITEPAPER.md

[12] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum yellow paper, 2014. Accessed: Oct. 2026. [Online]. Available: https://ethereum.github.io/yellowpaper/paper.pdf

[13] OpenZeppelin, "OpenZeppelin Contracts," official documentation. Accessed: Oct. 2026. [Online]. Available: https://docs.openzeppelin.com/contracts/

[14] Golem Factory GmbH, "Golem overview," official documentation. Accessed: Oct. 2026. [Online]. Available: https://docs.golem.network/docs/golem/overview

[15] Akash Network, "Akash: Decentralized serverless network," official repository and documentation. Accessed: Oct. 2026. [Online]. Available: https://github.com/akash-network/node

[16] Render Foundation, "Render Network whitepaper v4.0," 2023. Accessed: Oct. 2026. [Online]. Available: https://renderfoundation.com/whitepaper

[17] iExec, "Proof-of-Contribution," official protocol documentation. Accessed: Oct. 2026. [Online]. Available: https://docs.iex.ec/protocol/proof-of-contribution

[18] Protocol Labs, "Filecoin: A decentralized storage network," 2017. Accessed: Oct. 2026. [Online]. Available: https://research.protocol.ai/publications/filecoin-a-decentralized-storage-network/

[19] Y. Rao et al., "Bittensor: A peer-to-peer intelligence market," 2021. Accessed: Oct. 2026. [Online]. Available: https://bittensor.com/whitepaper

[20] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in Proc. 29th ACM Symp. Operating Systems Principles (SOSP), 2023. Accessed: Oct. 2026. [Online]. Available: https://arxiv.org/abs/2309.06180

[21] Ollama, "Ollama: run large language models locally." Accessed: Oct. 2026. [Online]. Available: https://ollama.com

[22] ShaderMesh project team, "ShaderMesh: Software requirements specification and phase plan," version 1.4, project repository, `docs/ShaderMesh-SRS.md`, Oct. 2026, unpublished.

[23] N. Johnson, "EIP-191: Signed data standard," Ethereum Improvement Proposals, 2016. Accessed: Oct. 2026. [Online]. Available: https://eips.ethereum.org/EIPS/eip-191

[24] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018. Accessed: Oct. 2026. [Online]. Available: https://www.w3.org/TR/WCAG21/

[25] National Institute of Standards and Technology, "Secure hash standard (SHS)," FIPS PUB 180-4, Aug. 2015. Accessed: Oct. 2026. [Online]. Available: https://csrc.nist.gov/publications/detail/fips/180/4/final

[26] Nomic Foundation, "Hardhat documentation." Accessed: Oct. 2026. [Online]. Available: https://hardhat.org/docs

[27] Ethereum Foundation, "web3.py documentation." Accessed: Oct. 2026. [Online]. Available: https://web3py.readthedocs.io

[28] Docker Inc., "Docker documentation." Accessed: Oct. 2026. [Online]. Available: https://docs.docker.com

[29] PostgreSQL Global Development Group, "PostgreSQL documentation." Accessed: Oct. 2026. [Online]. Available: https://www.postgresql.org/docs/

[30] J. Benet, "IPFS: Content addressed, versioned, P2P file system," arXiv:1407.3561, 2014. Accessed: Oct. 2026. [Online]. Available: https://arxiv.org/abs/1407.3561

[31] Vercel, "Next.js documentation." Accessed: Oct. 2026. [Online]. Available: https://nextjs.org/docs

[32] wevm, "viem: TypeScript interface for Ethereum." Accessed: Oct. 2026. [Online]. Available: https://viem.sh

[33] J. Moxey, E. Marcatilli, and R. Moore, "EIP-1193: Ethereum provider JavaScript API," Ethereum Improvement Proposals, 2018. Accessed: Oct. 2026. [Online]. Available: https://eips.ethereum.org/EIPS/eip-1193

[34] Microsoft, "Playwright documentation." Accessed: Oct. 2026. [Online]. Available: https://playwright.dev/docs/intro

APPENDIX A: DEPLOYMENT ADDRESSES AND REPRODUCTION

Table A.1 lists the Sepolia deployment (chain identifier 11155111, redeployed 2026-10-24). All six contracts are source-verified on Etherscan.

TABLE A.1: Sepolia contract addresses

Contract	Address
SmeshToken	`0x4d08515811A8b14A69aE3049278EDB04e83645F1`
ProviderRegistry	`0xd723DFd9E63632e9533ea1946b57AdB6d00cfC3f`
JobMarket	`0x9F3B0526A9c665F1Df423145eaACCe5b2479Ee57`
Verification	`0x994927B22613bCB8c5A25fA6386E386F250D365F`
RentalMarket	`0x0CcaB81947Ee0Dc270908FE0cd99f280596f3Ac3`
InferenceMarket	`0x108fF3eaeB81592A04212F52bEe50F46a0abf9D1`

The live provider is `0xfd4e4A034601991A3070bD2B9B0F6a473a5bfc76`. To reproduce the local system, use the commands below on a machine with Docker Compose v2, Node.js 20, and Python 3.11.

```
Copy-Item .env.example .env
docker compose up --build -d --wait
Invoke-RestMethod http://localhost:4000/health
docker compose exec -T orchestrator npm run smoke:inference

# Sepolia operator (requires a funded test key and an RPC URL)
cd contracts; npm ci; npm run compile; npm run deploy:sepolia
npm run verify:sepolia
cd ..; docker compose -f docker-compose.sepolia.yml up --build -d

# Additional provider laptop
python .\agent\discover_hardware.py
docker compose --env-file .\agent\.env -f docker-compose.provider.yml up --build
-d
```

APPENDIX B: API ENDPOINT SUMMARY

The orchestrator exposes the read API and the inference gateway listed in Table B.1. It never receives value writes.

TABLE B.1: Orchestrator API endpoints

Route	Purpose
GET /health	Service status. Returns 503 when persistence or indexing fails.
GET /listings, /leases, /leases/:id	Rental listings and leases.
GET /providers	Providers with stake, reputation, attestation, models, challenge summary.
GET /challenges, /challenges/:id	Benchmark challenges. Filter by provider.
GET /models, /offers, /offers/:id	Model catalog and inference offers.
GET /credits, /credits/:id	Prepaid credits. Filter by requester.
GET /jobs, /jobs/:id, /tasks, /disputes	Verifiable jobs, tasks, and disputes.
GET /jobs/:id/result	Final stitched artifact CID.
GET /events, /events/history	Server-sent event stream and recent indexed events.
POST /jobs/prepare	Split an image and pin the manifest. No chain write.
POST /v1/chat/completions	OpenAI-compatible gateway with an EIP-191 authorization.

APPENDIX C: TEST COMMANDS

Table C.1 lists the commands for each suite.

TABLE C.1: Commands to run each test suite

Suite	Command
Contracts	`cd contracts; npm ci; npx hardhat compile; npx hardhat test; npm run gas`
Provider agent	`cd agent; pip install -r requirements-dev.txt; python -m pytest`
Orchestrator	`cd orchestrator; npm ci; npm test; npm run typecheck`
Orchestrator with Postgres	Set `TEST_DATABASE_URL`, then `npm test`
Simulator	`cd simulator; python -m unittest discover -v; python verification_game_sim.py`
Web checks	`cd web; npm ci; npm run lint; npm run typecheck; npm run build`
Browser tests	`cd web; npm run test:e2e` (read-only), `npm run test:e2e:wallet` (wallet writes)
Regenerate this report	`python docs/_scripts/report_figures.py; python docs/_scripts/build_report.py`